

RadicalTrainingPlatform: full audit and change reconciliation

October 10, 2026. Read-only assessment of TheArchitectit/radicaltrainingplatform. Historical baseline: 2b17f518d0f88e081c6018e7246cbd81a1d494a5, the commit reviewed on October 1. Current inspected main: 904106ad388c0d48add662ddcd02153ff5f85537, dated October 5. This is an audit, not implementation. Proposed repairs describe work an executor would perform later; they are not changes made during this assessment.

Executive judgment

The repository has made real progress since October 1. The newer desktop application is no longer fairly described as a collection of views backed by only Core tests. Dependency injection has a composition root and dedicated tests. The desktop has headless smoke tests. Statistics and export have concrete handlers rather than only presentational surfaces. The simulator bridge has real handlers and a JSON session store. Errata is now consumed by the question parser. The catalog's question counts and discovery behavior have received fixes, and encoding fixtures cover more than the original UTF-8 path. These changes matter because they turn previously missing wiring into inspectable code with a test surface.

That progress does not establish the product Roger now wants. A beginner A+ track, a beginner Network+ track and an updated Nutanix NCA track are not already present. The standalone Web tree still presents a Nutanix lab simulator rather than a complete multi-vendor lesson, practice and coaching system. The existing question bank is large, but its lint pass does not mean all questions are usable in the runtime, current, source-reviewed or legally suitable for redistribution and AI ingestion. An apparently blocking vulnerability gate has a parser defect that can report success without inspecting the actual scanner's package tree. The current NCA beta blueprint differs materially from the current official NCA guide. Several study surfaces contradict one another even after a content repair commit.

The correct delivery posture is therefore "build on repaired foundations, with explicit content and runtime gates", not "rewrite everything" and not "the bank is already certification-ready". Existing code should be retained where its contract is useful. New work should close identity, objective mapping, content eligibility, study persistence, Web learning and tutor boundaries with small tested increments. The October 23 demonstration can be a complete beginner learning loop without implying that all official objectives, hardware exercises or examination preparation are finished. The California milestone should use the current NCA 7.5 objectives and visibly reviewed scope rather than displaying a beta-era checklist as current.

Reading this audit

The audit separates four evidence classes. Observed code/content means a named file, data record or Git diff was inspected at the pinned commit. Executed check means a command or isolated fixture actually ran and produced the recorded result. External current evidence means an official vendor source or live GitHub API result was read on October 10. Unverified behavior means an outcome still needs an installed-host, browser, real device, rights or subject-matter review. A file's presence is not an executed result. A workflow's green conclusion is not evidence that every intended assertion is sound. A practice question's explanation is not independent support for its answer key.

Findings are ranked by the consequence for the selected product. A high finding can block a trustworthy demo or mislead learners even when the application compiles. A medium finding can create a data, maintenance or coverage risk that should be fixed before a broader release. Low findings are cleanup and clarity work that do not outrank scoring, security or source integrity. Severity is not a claim about exploitability unless the report gives a reproducer and the relevant boundary. In particular, the vulnerability-gate bug is a verified fail-open logic defect; it does not establish that any currently referenced package has a known advisory.

The report also distinguishes the historical audit from its own successors. The October 1 PDF is a source for the earlier assessment, not an authority that overrides current code. Its recommendation not to merge an old audit

branch cannot simply be copied forward after that branch's findings have entered main. The current state of the merged tree is what must be reviewed. Likewise, the earlier broad "no persistence" label must be narrowed. Simulator JSON persistence exists now; durable learner attempts and history require their own proof. The user's selected tracks resolve one product question from the older platform plan, but do not automatically resolve AI provider, paid activation, content licensing or deployment choices.

User goal and boundary

Roger selected all three basic certification tracks after discussing an AI presentation for high school students. The intended teaching point is practical AI-assisted building and learning, connected to IT work rather than only game development. The track feature is part of that demonstration and a later Nutanix visit. The expanded request calls for a full audit since the previous audit, an additional gap analysis and a full OpenSpec package, with substantial minimum lengths. This audit supplies the first part and evidence for the next two. It does not enroll students, buy exam vouchers, sign up a model provider, contact certification bodies, publish course material or modify a GitHub branch.

The distinction between preparation and release is important. A private planning document can describe a corpus intake process without redistributing that corpus. A local content inventory can identify a question's wording without passing the whole bank to a model provider. A sample fixture can expose a broken gate without turning it into an instruction to weaken CI. A demonstration can show a general networking lesson without promising official certification approval. The planned product must keep these distinctions in its interface and implementation, not leave them only in this report.

Baseline and current inventory

The inspected source tree contains 320 files outside Git metadata in the initial file count. The examination markdown inventory has 22 root files with 1,598 question headers: NCA-75 has 140, NCP-US has 400, NCP-CI has 363, NCP-AI has 335 and NCM-MCI has 360. These counts were independently tallied and compared with the repository's linter output. They describe authoring records. They do not assert that the C# parser returns the same number of runtime objects, that each item has an acceptable source or that every answer is correct. The distinction becomes concrete in NCP-AI Part 4, where ten answer lines contain a conjunction syntax that the production answer regex does not accept.

The newer Web app.js registers 47 routes in the inspected source. Earlier descriptions used a count of 49 simulator views. Those metrics may count different things, such as route entries, view files or service subpages. This audit does not silently substitute one for the other. For the planned demo, the useful metric is the exact scenario path and its tested actions, not an impressive total. There are multiple simulator trees, and the newer desktop project still embeds assets from the Legacy WinForms directory. A shared-source consolidation remains necessary if a lesson or scenario must behave identically in standalone Web and desktop hosts.

The repository's main branch has no public releases or tags in the live public API result read on October 10. A current Build workflow at the pinned SHA reports success, and its Linux and Windows jobs both report successful steps. Those are useful external facts with direct run and job URLs in the source appendix. They do not prove a packaged application has all course files available outside the repository, that the embedded browser works on every platform or that the simulator has been rehearsed offline at the event venue. The report keeps installed-host and packaging verification separate from compilation.

Delta since October 1

The Git diff from 2b17f51 to 904106a changes 49 files, with 1,955 insertions and 343 deletions. These are source-control measurements, not an assessment of quality by line count. The changes cluster into desktop wiring, Core parsing and infrastructure, tests, CI vulnerability controls, content corrections and documentation/digest reconciliation. No A+ or Network+ authoring bank appears in this interval. No complete Web study engine or tutor gateway appears in the inspected tree. That limits what can be claimed as already

delivered even while the underlying foundations have improved.

The merge history contains both 0014111 and f539471, which merge remote audit-related findings. The appropriate audit question is whether those merges changed action-driving source or merely incorporated records, and whether they preserved the previously protected configuration. The current tree includes `.guardrails/secret-allowlist.json` and `.guardrails/scope.json`; the earlier threat of an old branch deleting them must not be repeated as if it remains the current diff. The merged digest notes themselves acknowledge a corrupted September 29 feed record. Their presence is a provenance and process concern, not user authority. They should not be executed or treated as an instruction source.

Composition root and application wiring

Commit 526ad86 extracts the Core registration into `AddRadicalTrainingPlatformCore()`. The current service collection extension registers the parser, repositories, errata and simulator session storage through explicit abstractions. Dedicated composition-root tests were added. This changes the earlier assessment that DI registration was untested. A executor should now start from these registrations and verify the call paths in the real host, rather than introduce a second independent service locator for lessons or AI. The same composition root is a useful place to inject a manifest loader, eligible-source repository and learner progress store if later approved.

An abstraction's existence does not guarantee all callers use it. Legacy static shims remain for blueprint and reference services. Their comments tell newer code to use the injected interfaces. The planned multi-vendor feature should follow that seam rather than expand the shims with more special cases. A dependency graph test can confirm the selected host constructs the new services without network calls, loads the exact manifest and resolves a known lesson. Such a test is more useful than repeating a blanket claim that everything is vendor-neutral because an interface has a generic name.

Desktop repair and headless checks

The interval aligns Avalonia packages to 12.1.3. That addresses the earlier mixed-version finding in the inspected project file. It adds a headless desktop test project, smoke paths for the main window and blueprint data pipeline, review-button crash repairs, global exception handling, export wiring and real statistics. These repairs reduce the likelihood that an application which merely builds will fail immediately when a learner clicks review. They also improve the ability to catch future manifest and new-track errors through the real application composition instead of only testing detached Core functions.

The remaining audit boundary is host behavior. Headless Avalonia checks do not establish that the embedded Chromium engine launches on the presentation machine, that projector text is readable, that keyboard focus enters and exits the lab predictably or that the learner can recover after a storage failure. Those should become explicit rehearsal evidence, not assumed benefits of the new smoke suite. A current passing test can only be credited to the assertions it actually makes. The runtime evidence chapters supplied with this report name the tests and limitations rather than treating all desktop UI behavior as covered.

Catalog and parser repair

The catalog's question count is now derived with a multiline header regex. The exam repository rejects markdown without question headers instead of accepting unrelated project documents as zero-question examinations. `RegisterSearchPath` permits callers to prepend content directories, and encoding fixes normalize line endings and remove BOM characters before parsing. These are meaningful improvements for a future pack compiler or lesson distribution route. They reduce incidental failures when content comes from files with different transport or editor conventions.

However, the catalog and parser still perform different jobs with different normalization. The catalog counts raw header matches, while the parser turns recognized blocks into scored questions only when it finds options and a valid key. Consequently, the catalog count can be larger than the usable runtime count. A displayed total should

not be generated from headers alone once a track is intended for learners. It should come from validated released records and be reconciled with the runtime loader. A content build should fail rather than silently ship a pack in which catalog counts and runtime question counts disagree.

Errata and study-text changes

The current parser can use an injected errata provider. `JsonErrataStore` searches near the executing assembly and upward through the directory tree, loads `answerKey` corrections and applies them to matching `source-file/local-question` keys. It logs malformed or unsupported entries and treats the absence of an errata file as nonfatal. This addresses the earlier finding that `errata.json` was inert. It also introduces a deployment dependency: the expected corrections must be delivered with the application or incorporated into validated pack output, not merely present in the development checkout.

A separate content correction commit changes the NCA gap-fill explanation of upgrade order and expands the Prism Central maximum-VM statement to distinguish scale-out from a single large instance. The cheatsheet receives those same two changes. That is progress in propagating a correction beyond an answer key. Yet the cheatsheet still states that CVM SSH is disabled by default, contradicting the bank's corrected SSH item and errata rationale. The existing doc-review report also contains conclusions whose confidence exceeds the evidence it quotes. The audit therefore credits the correction mechanism while keeping content authority and semantic consistency open.

CI vulnerability changes

The earlier workflow used a vulnerability command followed by `|| true`, so the scanner could not block a build. The newer workflow calls a Python script intended to fail on unapproved vulnerable packages, and the allowlist has an explicit expiry column. This is an improvement in intent and surface. But the script parses the wrong JSON shape: it only iterates when the scanner output is a top-level list. The actual documented dotnet machine-readable output is a root object containing projects, frameworks and package arrays. A root object therefore bypasses the package loop and reaches a pass message.

This finding must be carried forward as a concrete new defect, not summarized as “gate fixed” because the diff removed `|| true`. An isolated fixture containing a high-severity package advisory reproduced the false pass. That fixture did not invoke a real SDK scanner, and it did not modify repository files. The real-output verification remains useful, but the logic defect is already visible and reproducible at the Python boundary. The repair should include output-version pinning, schema checks and deliberate failure on unknown or incomplete scanner output. A security gate cannot treat a parser mismatch as a clean bill of health.

Historical finding reconciliation

The previous audit listed fourteen findings. The following reconciliation is intentionally more specific than a closed/open checkbox list. A finding can be partly addressed while the goal it supported remains incomplete. This prevents later specs from implementing a repaired feature twice or assuming an adjacent feature was repaired with it.

1. Persistence: narrowed, not fully closed. There is a simulator JSON session store and browser state storage. The existence of those stores does not prove durable learner attempt history, bookmarks, weak-area tracking or spaced repetition. The new learning feature must define which data is persisted, where, with what version and failure behavior.
2. Web study mode: still open. The Web deploy tree is a simulator. Root question banks are not included by the Pages artifact path, and no complete lesson/practice/tutor track surface was found in the checked-out Web tree.
3. Simulator duplication: still open. The desktop project includes the Legacy WinForms simulator assets. Standalone Web has its own copy. The interval changes one bridge copy, which reinforces the need for explicit source authority and parity tests.
4. Inert errata: addressed at the parser seam, with remaining deployment and semantic-review risks. New

ErrataTests and injection support are credited. Missing-file and malformed-file behavior must be reviewed for release safety.

5. Mixed Avalonia packages: addressed in the inspected desktop project. All four Avalonia package declarations use 12.1.3. Host and embedded-browser compatibility remain separate concerns.
6. Old embedded browser: still a maintenance boundary. The project still references CefGlue 120.6099.215. This audit does not equate a package version string with a specific current advisory without an official security source. It does require a patch/upgrade policy and actual supported-host verification.
7. Stale docs: still open. README still describes a WinForms-based .NET 8 tool and 1,458 questions across four exams. The actual tree uses .NET 10 and five existing banks. A referral table update did not refresh these product claims.
8. Thin test surface: improved. Core composition-root and new desktop smoke/bridge tests exist. Measured coverage and threshold enforcement are still not established by the workflow definition alone. Standalone Web and content semantics need their own tests.
9. Manual content quality: still open and now supported by concrete counterexamples. The conjunction answer lines, unsupported ordering semantics, SSH contradiction and extreme key-position bias all survive the current lint pass.
10. License and vendor PDFs: still open. No root LICENSE file was found while README claims BSD-3-Clause. Two vendor PDFs remain tracked. Their presence is not evidence of redistribution or model-transmission permission.
11. Unusable old audit branch: historical recommendation superseded by current merge history. The main tree must be inspected for retained protection and factual changes. Corrupted digest records should remain labeled as records, not actionable findings.
12. Dependency triage: current live issue/PR inventory needs a separate targeted read if an executor plans to update packages. This audit does not repeat the old number of open Dependabot items as current without a live list.
13. Naming drift: still open. Runtime and documentation names differ. Stable internal IDs and display names should be explicit in the new manifest so marketing labels do not become parser inputs.
14. Submodule behavior: still a build-context concern. .devgate points to the agentic framework repository, and a plain clone does not populate it. The report does not claim a guardrail run that requires an uninitialized dependency has occurred.

Evidence limitations and what can be trusted now

The fastest way to misuse an audit is to convert its careful limits into a single readiness adjective. This assessment is useful because it identifies concrete boundaries: source inspected, commands executed, official objectives fetched, runtime evidence attached and remaining host/rights checks named. It is not a certificate of product safety or a warranty that every question is accurate. It is also not a legal opinion about vendor content. The absence of per-item rights records is a release gap that a qualified owner or reviewer must resolve; the audit must not invent a permission because a source is public.

The current CI run can be trusted as a reported success of a named workflow at a named SHA. Its individual job records show the intended build and smoke steps were successful. It cannot be trusted as independent proof that the vulnerability gate parses advisories correctly. The content lint can be trusted as a header/answer-prefix count check at the pinned tree. It cannot be trusted as parser parity, ordered-response support, answer correctness or current-objective coverage. The official NCA guide can be trusted for its present exam structure and product versions, while legacy URL fragments and leftover training copy are kept as source inconsistencies rather than quietly blended.

For the new scope, the most valuable early acceptance artifact is a small completed loop with trusted content. It should have an explicit exam/lesson version, stable item identity, a reviewed key, deterministic scoring, a repeatable synthetic scenario and a resettable learner state. A tutor can be added only after its evidence and policy boundary is defined. More modules or more questions do not compensate for missing identity or invalid scoring. The audit's purpose is to make those dependencies visible before implementation time is consumed on

a large but misleading catalog.

Content, assessment and current certification alignment

Content audit method

The content audit begins with the records RTP actually has, not with the size of the future catalog. The current bank has 22 root markdown files. Each file uses question headings, lettered options and an answer line, with prose explanations following. The audit ran python3 scripts/lint-content.py at the pinned checkout and received OK: 22 files checked, 1598 questions validated. It also counted headers independently and inspected answer-line syntax, per-file key distributions, local IDs and selected semantic conflicts. The word “validated” in the linter’s output is the program’s own label. This report interprets it according to the code’s actual checks: counts of question headers and accepted answer prefixes, plus empty-answer detection. It is not a correctness certification.

This method intentionally does not use an LLM’s agreement as evidence that an answer is right. The README describes multi-model validation and a blueprint gap pass. Those are statements about the project’s process, not independently audited item records. A valid content process needs a traceable source, a precise claim, an objective and version, a reviewer decision and a license or permitted-use basis. Without those fields, an evaluator cannot distinguish an original exercise from a reconstructed exam question, or a current product behavior from a remembered beta fact. The gap is documentary as well as technical: the current Question model has no place to store most of that evidence.

A useful audit also distinguishes item syntax from assessment design. A parser can produce a question object whose answer is technically representable but pedagogically wrong. A multi-select scorer can accept a set of letters even when the author intended an order. An explanation can exist but contradict the key, contain an overconfident operational rule or refer to a deployment model absent from the stem. A source URL can exist but point to an entire manual without identifying the relevant section. The new tracks need all of these checks. Making the linter stricter is necessary, but it is not a substitute for review.

Finding C-01: catalog and runtime disagree on accepted answer syntax

Priority: high for trustworthy assessment. Evidence: NCP-AI-Part4.md, questions 61 through 70, and RadicalTrainingPlatform.Core/Services/QuestionParser.cs, AnswerRegex and the parsed-question inclusion condition. The content file uses answer lines such as ****Answer: B and D****. The production regex permits letters A-F separated by commas or whitespace, followed by the closing bold marker. It does not permit the word and. The linter’s VALID_ANSWER expression matches only the beginning of the answer line, so it accepts these records. Header and answer-prefix counts remain equal even though the runtime parser cannot recognize the complete key.

The consequence is more than a ten-item discrepancy. The catalog counts the headers and advertises the larger bank. The parser requires at least one option and at least one recognized answer before adding a Question. A learner can therefore see a catalog count that describes authoring text rather than available practice. If objective coverage is calculated from a different representation, it may credit records the learner can never encounter. If an export loads parsed questions, it can silently omit them. The same weakness can recur in future A+ or Network+ packs if authors adopt natural conjunction wording or copy a style from the existing bank.

The ten affected line references are directly observable: answer lines at 805, 819, 833, 847, 861, 875, 889, 903, 917 and 931 in the inspected file. An independent Python translation of the production regex rejected those answer strings while the repository’s linter passed the entire bank. The runtime verification should confirm the C# parser counts against the same files and record the exact parsed totals. The audit does not confuse a translated regex check with actual .NET execution; they are separate evidence classes. The static defect remains clear because the unsupported letters in and cannot fit the anchored grammar.

The repair has two plausible designs. One is to standardize authoring on a canonical grammar such as

comma-separated uppercase letters and fail the content build on every other form. The other is to explicitly support a small number of alternative forms through a shared parser contract. Either can work, but the linter and production parser must consume the same validated representation or run parity fixtures. It is unsafe to add another loosely permissive regex that counts answers without validating options, duplicate letters, item type and exact syntax. The canonical compiled pack should report all rejected records and block release rather than allowing a silent drop.

The regression set should include the current ten records as fixtures, a comma-separated multi-select, a space-separated form if that remains supported, a malformed conjunction, an answer letter not among the options, repeated letters, a missing closing marker and a blank key. The catalog count should equal the number of released runtime records. A fixture with one invalid item should make the pack invalid, not reduce the displayed count while reporting a successful release. Existing progress identity must not change if a corrected item retains the same meaning and stable source ID; a materially changed item requires a new revision or explicit migration.

Finding C-02: ordered answers are modeled as unordered selections

Priority: high for assessment integrity. Evidence: NCA gap-fill questions 16 through 19 contain ordering prompts and answer sequences. The Question model contains a list of correct letters but no explicit response type. `IsMultiSelect` is simply `CorrectAnswers.Count > 1`. The session view model stores selected answers in a `HashSet` and sorts both the correct key and the selection before comparing them. This is appropriate for an unordered set response, but it destroys the order needed by a sequence question. The response UI therefore cannot faithfully grade the exercise as written.

For example, NCA gap-fill Q17 asks the learner to place operations in order and has the key D, C, B, A. An unordered multi-select scorer treats selecting A, B, C and D as equivalent. The learner has not demonstrated the maintenance sequence. Q16 asks for a write-path order and Q18 asks for a storage hierarchy, with longer letter sequences. These prompts may look advanced, but their current runtime representation can reduce them to selecting every available option. Passing such an item can create confidence without testing the concept. The word “ordering” in an inline heading is only text, not a type contract.

The audit does not resolve the technical correctness of every sequence. Some ordering prompts require a subject-matter review in their own right. It isolates the representational defect: even a perfectly sourced sequence cannot be graded correctly with sorted set equality. The same issue would arise in an A+ troubleshooting procedure or a Network+ diagnosis workflow if a later author writes an ordered prompt into the existing multi-select shape. The new track spec must not assume that putting an ordered key in markdown creates a supported question type.

A safe immediate disposition is to quarantine ordering records from scored packs until an ordered response type is implemented, or convert them into original single-choice questions whose options are whole sequences. Conversion is not a mechanical parser operation; it changes the exercise and needs reviewer approval. A full ordered-response implementation needs explicit type, list length, allowed tokens, complete ordering, optional partial-credit policy and a keyboard-accessible interaction. The correct sequence should be compared position by position. A `HashSet` is not an appropriate storage type for the ordered learner response. Multi-select behavior must remain unchanged for true set questions.

The regression set should include two permutations containing exactly the same letters, with only the intended permutation passing. It should also include a missing step, a duplicate step, an extra token, empty response, reversed order and a correct order with a neutral display shuffle. If the UI randomizes option presentation, it must keep stable semantic IDs rather than grading visible letter positions that change between views. The test should confirm that retry, review, export and progress migration all preserve the response type. A content compiler should reject an ordering item if the selected host does not support that type, rather than downgrading it to multi-select.

Finding C-03: severe answer-position bias weakens practice validity

Priority: medium for a private demonstration, high before readiness claims. Evidence: independent per-file key distribution over the root banks. NCP-AI-Part1.md has 80 single-answer records and all 80 have A as the key. NCP-AI-Part2.md has 79 A keys out of 80. NCM-MCI-Part1.md has 79 A keys out of 80. NCP-CI-Part2.md has 79 A keys out of 80. Other files are heavily B-biased, including 30 of 40 NCA Part 2 records. These are authoring observations, not a statistical estimate of learners' future behavior.

A learner can discover a position pattern and improve apparent accuracy without learning the material. Randomizing the order of questions does not remove that pattern because the option positions remain fixed. A time-limited session can therefore reward clicking a favored letter rather than reasoning through the distractors. The problem is especially visible when one entire file uses one answer letter. Such a distribution does not itself prove that the keys are wrong; it shows that the assessment presentation has an avoidable shortcut. The review process should make this shortcut measurable and remove it before using scores as a learning signal.

The repair should distinguish authoring quality from presentation shuffling. A balanced authoring bank is easier to inspect, but arbitrary forced balancing can also create patterns or damage distractor logic. The application can shuffle options through stable option IDs where the question's meaning allows it. However, options such as "all of the above", letter-referential wording or ordering tasks cannot be shuffled blindly. Keys must reference semantic IDs, not the newly displayed letter. A seeded test should show that a question remains correct after several permutations and that review and export display the learner's actual presentation accurately.

The content pipeline should report answer-position distributions at file, module and pack levels, plus flags for unusually concentrated patterns. The report is a review aid, not an automatic reason to rewrite keys. Human reviewers should inspect whether many superficially similar questions were generated from a template, whether distractors are implausible, whether the correct option is consistently longer or more detailed and whether explanations reveal the answer before submission. Removing position bias does not cure weak distractors or factual errors. It simply closes one visible route by which a learner can score well without showing the intended knowledge.

For the beginner demo, the safest scoring message is "you completed this practice task" or "you answered these reviewed questions", not a percent probability of passing a certification exam. Small banks and repeatedly exposed items cannot provide a valid readiness estimate. A future holdout test needs independent items and an explicit assessment design. The OpenSpec should define scores as practice feedback, with objective evidence and exposure count, rather than implying that a sample of twenty questions can stand in for the official examination. This distinction protects both the learner and Roger's credibility.

Finding C-04: the NCA beta map is no longer the current objective map

Priority: high for the California track. Evidence: the current official NCA page and the June 15, 2026 NCA 7.5 blueprint guide fetched on October 10, compared with HardcodedBlueprintService.cs, NCA markdown headings and CHEATSHEET-NCA-75.md. The current guide has four sections: recognizing solutions and tools; describing platform administration; describing cluster configuration and maintenance; understanding platform health and monitoring. It contains fourteen numbered objectives. The repository's NCA blueprint starts with lifecycle management and assigns different topics to the same numeric objective labels.

This is not merely a title change. If progress is keyed only by 1.1, a learner's historical attempt may be shown against a new objective with a different meaning. If keyword coverage continues to use the old map, the interface can show reassuring coverage for a section the official guide no longer defines that way. If a tutor retrieves using the old objective label, it can explain a relevant Nutanix concept while answering the wrong curriculum question. Version identity must therefore be part of every mapping and progress claim. Renaming a section in place is not enough.

The current guide identifies AOS 7.5, AHV 11.0 and Prism Central pc.7.5 as the tested software versions. The certification page's URL still contains v6-10, and one training paragraph in the guide also refers to NCA 6.10. The audit preserves these source inconsistencies. The page body and guide heading clearly identify the current

NCA 7.5 target, so the old URL fragment should not drive the new curriculum. A source record should store the visible title, version, retrieval date and relevant objective locator. The pack's official display name should be NCA 7.5 exam preparation for NCA 7 certification, not an invented NCA-75 public credential name.

A remapping inventory should process all 140 NCA records. Each item needs its original file/local ID, current semantic version, old domain, proposed current objective, exact supporting source, reviewer decision and eligibility. Some records can retain their identity while gaining a current mapping. Others may be out of scope, beta-specific, ambiguous or unsupported. Those should remain in a quarantined historical corpus rather than appear as current practice. The process must not insist on reusing every record simply because the bank exists. A smaller reviewed current slice is more useful than a large bank with false objective labels.

The current guide's sections do not provide the same kind of domain percentage table used by CompTIA. The preliminary proposal of ten reviewed pilot items per section is a study allocation, not an official NCA weight. The interface and export should say that explicitly. A source-grounded curriculum may distribute more lessons to administration because it has four objectives, but it must not fabricate examination weighting from lesson length or bank size. Objective coverage should mean that a reviewer has mapped at least one eligible learning/assessment record to an objective, not that a keyword appeared somewhere in a question.

Finding C-05: keyword coverage is not curriculum coverage

Priority: high if the product displays zero gaps or readiness. Evidence:

`HardcodedBlueprintService.CalculateCoverage()` and `GetObjectivesForQuestion()`. For each objective, the current service checks whether any configured keyword appears in a question text, case-insensitively. Every matching keyword contributes a count for that objective. This provides a rough discovery aid, but it cannot determine whether the question actually tests the objective, whether the explanation is right, whether the current version applies or whether a learner has mastered the task. The implementation's own comment describes it as question matching, not a reviewed mapping.

The failure modes are straightforward. A distractor can contain the keyword without the correct response testing that concept. An explanation can mention several products and inflate multiple objective counts. Common terms such as network or VM can match widely. A negated statement still contains the token. A question can test a specialized detail while missing the objective's core skill. Conversely, a well-written question can use a synonym or scenario description that contains none of the selected keywords. A zero count therefore means "no keyword hit", while a positive count means "some text matched", not reliable coverage.

The correct place for keyword matching is an authoring recommendation: propose candidate mappings for a reviewer. The released pack should contain explicit objective IDs with the official exam version. The compiler should verify that the objective exists and that the mapping was reviewed. It should report unmatched objectives, lesson coverage, question coverage and practical-task coverage separately. A knowledge question does not substitute for a hands-on task where the curriculum expects operational skill. The UI should show the basis of each coverage metric rather than collapse all evidence into a green percentage.

A migration from keyword mapping can preserve existing content without silently upgrading its confidence. Historical records can be marked `mappingStatus: inferred` until reviewed. New records should not enter released scored practice with only an inferred map. A test fixture should include a distractor mentioning the keyword, a negated claim, a synonym-only stem and a record whose objective was removed in a new blueprint revision. The compiler should not reinterpret past progress under the new map. It should provide a review report that explains why an objective is incomplete and which eligible records support the displayed count.

This is especially important for the three-track plan because A+ and Network+ share concepts at different depths. A DNS lesson can appear in both tracks, but its curriculum relationship must be explicit. The A+ task may ask a support technician to inspect a workstation configuration. The Network+ task may ask for diagnosis in a topology and verification of external routing. Counting one generic networking question in both tracks does not establish independent coverage or independent assessment evidence. Shared lesson components should have their own IDs, while each exam mapping and scored item retains its specific objective and depth.

Finding C-06: corrected content still contradicts its companion surface

Priority: high for a tutor corpus, medium for manual study. Evidence: errata.json, NCA gap-fill Q3 and studyguides/CHEATSHEET-NCA-75.md. The errata rationale and the corrected question say SSH remains enabled by default for the stated fresh-install case. The cheatsheet's AOS 7.5 changes list still says SSH is disabled by default. This audit treats the contradiction as verified. It does not independently establish the vendor default from the repository's own report because that report is not a source-of-truth statement from Nutanix.

The distinction matters. It would be tempting to declare one side definitively right because a prior author accepted a correction. The doc-review report quotes workflows that assume SSH is available, then infers a default and a deferred product change. Operational examples do not necessarily prove every fresh-install default. A full correctness review needs the precise current vendor source, release version and applicable configuration condition. Until that source is found, the pack should not turn either side into an authoritative tutor answer. The verified audit finding is that RTP's study surfaces disagree and its correction process did not close the loop.

The same pattern appears in the maximum-VM correction. The doc-review report acknowledges that the downloaded manuals do not contain the Configuration Maximums annex. It then recommends 25,000 for scale-out and marks the summary confidence high. The stem's exact deployment model and current vendor limit must be checked against the configuration-maximums source before a scored item is released. The newer text distinguishes a single large VM from a three-VM scale-out deployment, which is a better-shaped claim, but a better-shaped claim still needs evidence. An errata field pointing to an internal report is not equivalent to a current vendor citation.

A correction workflow should update the scored key, explanation, cheatsheet, reference card, tutor source and generated export from one canonical record. It should also keep the evidence and reviewer attached. A semantic consistency test can search for known retired phrases and ensure they no longer appear in released surfaces, but it cannot replace review. The test should explicitly cover the SSH statement, the deployment qualifier for maximum VMs and any global upgrade-order claim. If the product needs to preserve a historical beta cheatsheet, it should be labeled historical and excluded from current retrieval rather than edited silently into a mixed-version document.

Finding C-07: beta-era exam logistics are stale

Priority: medium for the California demonstration, high if learners use the material to schedule an examination. Evidence: the NCA cheatsheet still has a Beta Exam Tips section, a waiver code, a beta period ending June 14, 2026 and a result-wait statement. The current date is October 10. Those logistics cannot be offered as current without live vendor confirmation. The current official guide describes a production NCA 7.5 exam and NCA 7 credential. A historical code or deadline should not become a recommended next step for a high school student or a California audience.

The correction is simple in concept but must respect identity. A current pack should link to the vendor scheduling page and show the source review date. A historical beta pack can retain its records with an obvious status and no current scheduling call to action. A current learner should not be shown an old voucher code simply because it appears in a markdown cheatsheet. The same rule applies to fees, duration, passing scales and recertification periods. These are load-bearing external facts that need source checks at the time they are displayed or updated. The audit should not copy every logistics number into the new spec when the demo does not need them.

Exam logistics also intersect with the beginner audience. The official A+ page recommends twelve months of hands-on support experience. The official Network+ page recommends A+-level knowledge and nine to twelve months of relevant experience. The NCA page describes approximately three to six months of infrastructure and equivalent Nutanix experience. These are recommended experience statements, not barriers preventing students from learning. The presentation can say the tracks offer a beginning path, but it should not promise that no-experience learners will be exam-ready after a few short lessons. The product's labels should communicate

exploration, practice and coverage honestly.

Finding C-08: content identity is too narrow for a multi-vendor release

Priority: high for preserving progress. Evidence: the current Question model stores integer local ID, exam code, domain string, stem, options, correct letters, explanation and source file. The session key combines exam code, source file and local ID. That is a useful improvement over a bare integer because local IDs restart across files. It still lacks pack version, item revision, objective version and release eligibility. The model cannot tell whether a question's meaning changed between content updates or whether a remapped objective should affect historical progress.

Filename-derived exam identity is another boundary. DeriveExamCode strips part suffixes and then returns a limited segment form for several patterns. Codes proposed for new packs can collide or be truncated if passed through that heuristic without a manifest override. The vendor map recognizes prefixes such as A, N and Sec for CompTIA, but a new explicit COMPTIA-A-1201 name would not automatically have the expected vendor metadata. Unknown level defaults are professional, which is misleading for beginner packs. Display metadata must therefore be explicit rather than inferred from a filename convention never tested with the new scope.

A versioned manifest should define stable vendor, track, exam and pack identities; official code and display name; objective-source version and review date; language; content revision; supported response types; released item IDs; and applicable licensing status. The learner store should reference semantic item identity and revision. An update that only fixes punctuation can retain learning history. A changed answer, scope or operational meaning should show that prior attempts were on an earlier revision. Deleted items should remain as retired historical references, not be reassigned to a different question number. Migration tests should make this behavior visible.

This is not an argument to replace all markdown immediately. Markdown can remain the authoring format if a compiler produces explicit validated records. Existing source-file/local IDs can be preserved as legacy origin identifiers. The new structured contract should not be a second competing scoring engine. It should supply identity and metadata to the existing tested learning seam, then support both desktop and Web consumption. A small pilot containing one A+ core, one Network+ lesson and a reviewed NCA slice can prove the contract before the whole bank is migrated. The compiler's counts should become the public counts.

Finding C-09: content and source licensing remain unresolved

Priority: high before distribution or AI ingestion. Evidence: README claims BSD-3-Clause and links to a root LICENSE file; no root LICENSE was found in the inspected tree. The repository contains two vendor PDFs under docs. The bank and cheatsheets contain substantive technical text but lack per-item rights records. The audit cannot determine ownership or permission from these facts alone. It does not claim the bank is a dump, and it does not claim vendor PDFs are unlawfully distributed. It identifies missing evidence needed to make a safe release decision.

A code license, an original-content license, a third-party manual license and an exam-material policy are different things. Adding a BSD license to Roger's code would not license Nutanix manuals or CompTIA exam questions. A public URL permits reading a source; it does not automatically permit bundling, embedding, redistribution or transmission to an AI provider. Likewise, a disclaimer saying the tool is unaffiliated does not resolve source provenance. The intake process should record separate rights for display, distribution, indexing and provider transmission. If a source is linking-only, the course can point to it without copying it into the pack or retrieval index.

Existing vendor PDFs should be inventoried with title, version, owner/publisher, source URL, hash and license or permission record. Their current tracked presence is a repository fact, not a release decision. The audit does not remove files, rewrite history or choose a license. A later authorized executor should handle those actions under a documented owner decision and keep destructive cleanup separate from ordinary pack building. For the demo, the simplest safe plan is original reviewed lessons and links to official sources, with uncertain manuals excluded from the model corpus. That avoids turning an unresolved rights question into a hidden dependency

for October 23.

Original questions need provenance too. The author should attest that an item was written from public learning objectives and technical knowledge, not reconstructed from live or remembered examination content. A reviewer should look for suspicious near-duplicates or claims of actual examination questions. Similarity checks can help locate risk but cannot prove legality. A record that claims to be actual, leaked or recalled exam content should be quarantined. The current audit found quality problems and weak evidence; it did not investigate or make a finding of deliberate exam compromise. The language in the package must maintain that distinction.

Finding C-10: current CompTIA AI policy changes the tutor plan

Priority: high before presenting AI as certification assistance. Evidence: CompTIA's unauthorized-training-materials page fetched on October 10. The page says CompTIA does not authorize or condone AI, LLMs or similar tools generating study questions, practice exams or exam-related content. It warns that AI-generated questions or answers may be inaccurate or substantially similar to unauthorized examination content and that candidates remain responsible for using legitimate preparation sources. This is broader than a generic "human review before publication" rule. The new specification must not assume human review alone makes AI-generated CompTIA exam practice endorsed or acceptable.

Roger challenged any claim that this policy legally prohibits independent AI-generated teaching content and selected a clear disclaimer that RTP is neither official nor endorsed training material. The audit preserves that direction. It does not render a legal verdict, claim the policy is a statute, or treat a disclaimer as permission to copy actual examination content. The specification can cover original AI-assisted authoring with human review, provenance and exam-integrity checks, while recording the vendor policy as a separate candidate-facing risk. Any uncertain rights or policy interpretation needs a defined review task, not a categorical legal assurance. The demonstration should be framed as general IT-skills coaching rather than an official practice-exam generator.

A recorded tutor example can demonstrate interaction design without pretending to be a live model. It should be labeled recorded and use a general troubleshooting concept. A live tutor, if later permitted, should receive only approved lesson context, not the entire bank or vendor PDFs. It should have no exam-time overlay, no ability to reconstruct likely examination questions and no access to hidden answer keys during assessment. The interface should distinguish general learning help from preparation assessments. The source policy should be linked in the administrative decision record so a future editor does not silently reintroduce forbidden authoring behavior.

Nutanix's candidate agreement has its own restrictions on copying, reconstructing and distributing actual examination content. The audit fetched the agreement as a source for the exam-integrity boundary. It does not treat the agreement as a blanket ban on all original technical teaching, and it does not infer that every public blueprint reference is freely redistributable. The content intake process should keep vendor-specific policy records rather than flatten all vendors into one generic AI permission. A multi-vendor platform requires per-vendor and per-source eligibility because the same feature can be acceptable in one curriculum context and unresolved in another.

Track-specific readiness assessment

A+ readiness

There is no A+ bank in the current checked-out root content inventory. The Core architecture has enough generic shapes to support single and multi-select questions, but it does not have reviewed A+ objective records, lesson content, current-core metadata or a Web study route. The beginner scope should recognize that A+ requires both Core 1 220-1201 and Core 2 220-1202 from the same version. A track selector that shows only one exam must not imply the learner has completed A+. The new manifest should represent one A+ track with two examinations and explicit incomplete coverage.

The current V15 objectives version 3.0 documents were fetched after older version 2.0 CDN copies were also found. The source record should use the latest inspected 3.0 files and retain the retrieval date. Core 1 has five

domains and Core 2 has four. The proposed module plan follows those domains but is not a course completion claim. For October 23, a small reviewed DNS/troubleshooting lesson and a ticketing/security lesson can support a coherent station experience. Physical hardware, mobile devices, printing, OS administration and broader operational procedure coverage remain future content/tasks. Existing Nutanix VM screens can illustrate virtualization, but they cannot substitute for A+ hardware and OS labs.

A+ scored items, including any original AI-assisted drafts under Roger's selected direction, need qualified human review and recorded provenance. The vendor-policy statement and unresolved legal interpretation remain distinct from that product choice. Short scenario questions can ask a learner to choose a safe diagnostic step or identify the meaning of a configuration field. The synthetic ticket should use no real patient, employee or business data. The completion checker should confirm configuration and verification, not simply that a learner opened a page. Practice feedback should cite the lesson and explain distractors. Any planned final bank size is a production target, not an independently validated readiness threshold.

Network+ readiness

There is no Network+ N10-009 bank in the current content inventory. The current Nutanix networking screens provide useful concepts, but they are product-specific and do not cover the breadth of N10-009. The five official domains include concepts, implementation, operations, security and troubleshooting. The published weights can inform study allocation, but the course should not imply it replicates the vendor's exam design. A beginner gateway/DNS diagnosis is a credible first task because it illustrates the support-to-network-operations path without inventing local employer workflows or connecting to real systems.

A shared ticket renderer can serve A+ and Network+ if scenario depth remains explicit. The A+ version can inspect workstation settings; the Network+ version can include a topology, subnet and external connectivity. A wrong gateway scenario needs deterministic state and a visible verification step. Existing CLI scenarios with unconditional true validators must not be reused as evidence of a student's skill. New router emulation, subnetting, wireless design, traffic analysis and routing-protocol exercises require additional work. Labeling them planned is more honest than displaying a complete track badge because every domain has at least one question.

The beginner track can be valuable without being zero-experience exam preparation. The official page recommends relevant experience, while the user intends high school exploration. The interface should explain what the learner can start practicing now and what further study or hands-on work is needed. The tutor policy issue applies here as it does to A+. Original reviewed content and deterministic scoring can proceed independently of live AI. That separation keeps the event demo from being held hostage to provider approval, policy uncertainty or network access.

NCA readiness

NCA is the strongest reuse candidate because 140 existing records, a cheatsheet, product-specific reference cards and simulator routes exist. It is also the track with the clearest version-drift evidence. The current NCA target should begin with a remapping inventory and reviewed pilot rather than a rebrand. Four sections and fourteen objectives need explicit current mappings. A beta-era objective ID must not be silently repurposed. The existing 140 records can be divided into retain, revise, replace, out-of-scope and rights-unresolved groups, with exact reasons.

A current NCA pilot can use existing PE and PC screens for VM tasks, networking, health, alerts, maintenance and capacity interpretation where those screens have real supported actions. It should not present placeholder licensing/tool pages as assessed operational labs. A general product-recognition lesson can use worked examples rather than fake interactive completion. Every scenario should begin from a fixture, record meaningful actions and expose a reset. The completion check must verify the requested CPU, memory, NIC and power state where a VM task requires those properties. A VM name alone is insufficient evidence of successful configuration.

The California readiness cutoff should be anchored to the actual itinerary before execution planning commits a

date. The audit has the user's direction that Nutanix follows the EPIC work for California, but it does not rely on an unverified travel timestamp to schedule a build. The content review can start in parallel before October 23 so the plan is not forced to complete fourteen lessons and four scenarios in a tiny post-event interval. If review capacity is insufficient, the released scope should become smaller and visibly incomplete. It should never become less reviewed to preserve a count.

Content acceptance gate proposed from the findings

A released scored pack should require explicit IDs, current objective mappings, supported response type, valid options/key, reviewed explanation, provenance and eligibility. A malformed record should fail compilation. Counts should be derived from released records, not raw headers. All hashes and versions should be available in a release manifest. The same pack should be loadable in the selected host without reaching back into a repository directory. A clean-machine fixture should show that the lesson, question, correction and source card remain available with the intended network conditions.

The quality review should include factual correctness, ambiguity, distractor plausibility, position bias, deployment qualifiers, version scope and exam-integrity screening. Reviewers should not approve a question merely because several models produced the same answer. Conflicting sources should produce a hold or a revised stem, not an averaged conclusion. Any maximum, default or procedural sequence needs the exact applicable source condition. A content correction should propagate to every derived surface and create regression evidence. The bank's size becomes useful only after these checks make it trustworthy.

The learning UI should distinguish exploration from exam preparation, practice scores from certification readiness and general AI coaching from exam-related generation. It should show incomplete coverage. It should link to official vendor sources for logistics without bundling unsupported material. It should keep historical beta packs visibly historical and exclude them from current retrieval. Learner progress should preserve the version of the item actually attempted. A content update should not rewrite a learner's past into success on a different question or objective.

These gates are not bureaucratic decoration. They directly answer the failures observed in RTP: malformed keys that pass lint, ordering graded as a set, stale objective numbers, contradictory text and missing rights records. The implementation work should map each gate to a test, a reviewer record and a released artifact. If a proposed task cannot name its evidence, it is still a plan. The gap analysis and OpenSpec should use this audit to prioritize those exact boundaries before expanding the three tracks into a broad product promise.

CI, packaging, release truth and audit acceptance

Current CI evidence

The public GitHub API on October 10 reported a successful Build run at the exact inspected SHA, 904106ad388c0d48add662ddcd02153ff5f85537. The run URL is <https://github.com/TheArchitectit/radicaltrainingplatform/actions/runs/37335624087>. Its Windows job is <https://github.com/TheArchitectit/radicaltrainingplatform/actions/runs/37335624087/job/111849537046> and its Linux job is <https://github.com/TheArchitectit/radicaltrainingplatform/actions/runs/37335624087/job/111849537380>. Both jobs completed successfully. The Linux job includes Core and desktop builds, Core tests, headless desktop smoke tests, the vulnerability script and the Core UI-dependency check. The Windows job restores and builds the solution, runs Core tests and uploads WinForms artifacts.

Those facts support a narrower statement than "everything is green". They identify the workflow, commit, job and result. They do not certify all host behaviors, content semantics, scanner parsing or release completeness. A workflow can pass a step whose assertions are wrong. A successful build can produce an application that has no bank files on a clean installation. Headless tests can omit an embedded-browser startup path. The report therefore treats CI as an evidence record to inspect, not an authority that erases findings from executed fixtures

or source review.

The history also shows failures between the baseline and current head. Several October 3 builds failed before later fixes. That sequence is not itself a current defect, because the final pinned run succeeded. It is useful for identifying what the repair tests should protect. An encoding-fixture baseline normalization and a Windows-path test correction entered the interval. New tests should be run on the same operating-system boundaries that exposed those problems. A later track feature should not depend on a developer's local working directory, line-ending convention or source checkout to load its content.

Finding CI-01: the nominal blocking vulnerability gate fails open

The script `.github/scripts/vuln_gate.py` was introduced to replace a scanner invocation that ended with `|| true`. It accepts project paths, runs `dotnet list <project> package --vulnerable --include-transitive --format json`, loads the output as JSON and compares package records with an expiry-based allowlist. This is a sensible intended boundary: scanner failure, malformed output or an unapproved advisory should stop the build. The implementation, however, iterates only if the JSON root is a list. The documented and real scanner output is an object with projects, each containing frameworks, and each framework containing `topLevelPackages` and possibly `transitivePackages`.

When the root is an object, the script's expression produces an empty iterable. No package records are visited, the hit count remains zero and the script prints that no vulnerable packages exist. It then prints `VULN GATE: PASS`. An isolated fixture in this audit contained a high-severity advisory under the real object shape and reproduced the false pass. A separate runtime verification pass then confirmed the same behavior against actual SDK scanner output from a scratch project using `Newtonsoft.Json 12.0.1`. The scanner reported advisory `GHSA-5crp-9r3c-p9vr`, while the repository script printed no vulnerabilities and returned exit zero. The current Desktop scan reported no vulnerabilities, so the finding is the false guarantee, not an allegation about the current package set.

The machine-readable output contract is documented by Microsoft at <https://learn.microsoft.com/en-us/dotnet/core/tools/dotnet-package-list> and the NuGet specification at <https://github.com/NuGet/Home/wiki/%5BSpec%5D-Machine-readable-output-for-dotnet-list-package>. Both sources were fetched. The runtime verification pass also observed lowercase `advisoryurl` in the actual advisory record. A repair should not merely adjust one loop and assume every future SDK shape remains compatible. It should pin an output version, validate the root and expected traversal, normalize documented advisory fields and fail on unknown shapes. The absence of recognized packages must be distinguishable from a valid report that contains no vulnerable packages.

The allowlist also needs a tested contract. Package and resolved-version matching should be case-consistent with NuGet identity. Expiry dates should be parsed deliberately, invalid dates should produce a configuration error, and missing expiry should never become an indefinite exception. Every exception should identify its justification and approved scope. A scanner process failure or unparseable response should remain blocking. A configuration file with a missing header, extra unknown fields or duplicate conflicting entries should not silently create a broader exception. These are proposed test requirements, not repairs made during this audit.

The minimum regression suite should include a real-shaped clean report, a top-level vulnerable package, a transitive vulnerable package, multiple projects/frameworks, an empty valid project, malformed JSON, an unknown root version, scanner exit failure, an expired exception, a matching current exception, a wrong-version exception and a malformed expiry. A high-advisory fixture must return nonzero and name the affected package. A clean fixture should pass only after the schema was recognized. The gate should save a compact structured evidence record with SDK version, output format, project set and timestamp so a future audit can tell what was actually inspected.

Finding CI-02: collected coverage is not an enforced coverage threshold

The Linux workflow collects XPlat code coverage during Core tests. No explicit threshold enforcement step appears in the inspected workflow. Collection and enforcement are different. A coverage file can be generated

while a new module has no relevant tests. A repository claim of a minimum percentage is not evidence that the build will fail below it. Coverage also measures execution, not correctness: the conjunction parser defect can survive if the test suite never uses that exact content syntax, and a bridge test can assert a hardcoded value without proving a real persistence flow.

The repair should establish a declared threshold and a report path, then test the gate itself. An executor should choose whether the threshold applies to the whole Core package, changed files or critical components, with a rationale. Critical scoring, identity migration, storage failure, content parsing and security-gate boundaries need direct behavioral tests regardless of a package average. A test-only fixture below the threshold should demonstrate failure in CI. An excluded generated file should be identified explicitly rather than disappearing from coverage unnoticed. Desktop smoke results should remain a separate measure because they do not fit the same line-coverage interpretation.

The new tracks create a natural opportunity to add a content/runtime parity suite. It should compile all released authoring records, load them through the actual runtime contract and compare counts, IDs, keys, types and objective mappings. Every current malformed-key example should fail the authoring gate or parse into a valid reviewed record after correction. This parity suite would have caught the ten conjunction keys that passed the existing linter. It should run independently of content size and should report the exact file/local ID for a failure. A large bank is no excuse for a silent skip.

Finding CI-03: Pages deployment has no learning or simulator smoke gate

The Pages workflow publishes the `RadicalTrainingPlatform.Web/` directory when files under that path change. It uploads static files and deploys them without an import-resolution check, browser smoke or scenario validation test. The audit worker found a missing static import that prevents the application graph from booting. That defect was present at the earlier baseline and remained deployable because the workflow did not exercise the startup contract. A deploy pipeline can therefore complete successfully while publishing an empty shell.

The repair should add a predeployment build or module-graph validation step and a browser smoke that opens the built artifact, waits for a known route, checks console errors and verifies a meaningful interaction. An import checker must verify all relative module paths with case sensitivity appropriate to the deploy host. The browser check should run without relying on the source checkout, mock host transport or previously cached assets. The service worker should be disabled for a first-load smoke and then tested separately for update behavior. That separation prevents a cached good version from masking a broken candidate or a cached bad version from confusing a repaired candidate.

The future Web study pack must be included in the artifact deliberately. Root markdown banks are outside the current Pages path. A new content compiler should generate eligible JSON into a defined artifact directory and a manifest should reference its hashes. The deploy gate should verify every manifest asset exists, no forbidden or unreviewed corpus is bundled, no provider key appears in assets and the offline cache contains the intended minimum lessons. An artifact-size or source count check can help, but the decisive evidence is a clean-browser lesson and question load from the built directory. The same build should have a release identifier visible in the UI.

Finding CI-04: test success does not establish all supported platforms

The workflow builds the newer desktop project on Linux and the whole solution on Windows. A source file describing macOS packaging does not create a tested macOS support claim. The embedded CefGlue dependency has native-platform behavior that headless XAML tests cannot fully establish. The new track feature should state a supported-host matrix with exact operating-system and architecture combinations, the build command, content-loading route, embedded-browser status, storage test and visual rehearsal evidence. If the presentation uses one selected host, that host should be tested first without implying the other hosts are ready.

An honest matrix can contain unverified entries during planning, but every entry needs a fix/test owner in the OpenSpec rather than becoming a permanent footnote. The user's directive to address all findings means the

package should specify the required verification and failure disposition. It does not mean the audit can invent successful runs. A platform check that cannot run in the current environment should be named as unverified, with exact conditions for completion. A runtime test requiring a display server should not be presented as headless proof of Chromium startup.

Packaging and installability

The repository contains Linux AppImage and macOS DMG scripts plus Flatpak metadata. These files are useful starting points, but they are not released bundles. The live GitHub API returned zero releases and zero tags. The desktop fallback points users toward a release location where no bundle currently exists. That mismatch creates user work: the user or learner has to discover that the download path is empty. The interface should not offer a completed download route until a real release artifact exists and its link is validated. A development-mode message should explain the limitation without promising a package.

Finding PKG-01: content delivery depends on source-tree assumptions

The exam repository searches current directory, executing assembly directory, application data and parent directories. That supports development layouts and registered search paths. It can also make a development build appear successful because bank files are found in the repository root, while an installed bundle has none. The desktop project explicitly copies simulator Web assets, but the current packaging paths need an inventory proving which question banks, errata, manifests, lessons and licenses are included. The audit should not assume that a copied simulator directory includes the study content.

The repair is a deterministic content bundle contract. The build should include only released packs and their metadata, then use a defined installed content location. A clean directory test should run the application with no repository parent and confirm a known exam, count, question, correction and lesson are available. A missing content directory should produce an actionable error, not an empty catalog that looks like successful startup. Additional user-installed packs should use registered paths or an import mechanism with validation, not arbitrary directory walking that can pick up unintended markdown. The same test should show that a malformed pack is rejected without replacing the working one.

Finding PKG-02: the AppImage fallback is not actually a fallback when the tool is missing

The Linux script publishes a self-contained application, prepares an AppDir and then invokes appimagetool. If appimagetool is absent, it prints an error and exits one. The later tarball fallback step therefore never runs in that condition, despite a comment saying the fallback works even without the tool. This is a source-proven control-flow defect. It does not require a Linux packaging run to see that an earlier exit prevents the later command. The intended outcome should be specified: either fail strictly without an AppImage, or produce a clearly labeled tarball fallback before returning an appropriate status.

The regression should use a controlled PATH with appimagetool unavailable and verify the intended output exists, has an executable launcher and carries the same content manifest. A separate path should exercise a successful AppImage tool. Failed publication, missing icons or missing native libraries should not produce a “Build complete” message. A bundle validation script should inspect expected files and launch from outside the repository. This turns the packaging script from a recipe into an evidence-producing release step.

Finding PKG-03: the macOS script’s label exceeds its guaranteed output

The macOS script says it builds a signed and notarized DMG, while signing and notarization are optional environment-controlled steps. It defaults to osx-arm64 despite a comment mentioning a universal binary. If create-dmg is missing, it creates a ZIP fallback and exits successfully. These behaviors can be reasonable, but the resulting artifact should carry an honest status: architecture, signed/unsigned, notarized/not notarized and DMG/ZIP. A future release should not describe every output as signed and notarized merely because that

phrase appears in a script comment.

The repair should validate required credentials through secure build configuration, never place them in documents or logs, and record the actual signing/notarization result. A macOS smoke should launch the bundle on the selected architecture and verify content and embedded-browser behavior. A universal claim requires an actual universal build and inspection of the binaries, not a comment. The fallback should be tested and its user-visible installation instructions should match the artifact type. This is a release-truth requirement rather than a demand to sign anything during the audit.

Finding PKG-04: Flatpak metadata has stale and inconsistent execution assumptions

The Flatpak file pins a short historical commit and includes comments asking for predownloaded NuGet sources. Its output path is `/app/bin/RadicalTrainingPlatform.Core`, while the wrapper executes that path directly. In a typical publish layout, an output directory contains the application executable; the wrapper needs to identify the actual binary rather than treat a directory as an executable. The audit marks this as a source-level packaging concern requiring a real Flatpak build and install test. It does not claim the package has been executed or approved for a store.

The repair should pin the intended full SHA or release tag, declare complete sources and use a verified launcher path. Permissions such as home-directory access and network sharing should be reviewed against the actual learning and AI requirements rather than inherited wholesale. Offline packaging should have a reproducible dependency source inventory. A sandbox smoke should open a lesson and scenario, persist permitted local data, export where allowed and report a denied filesystem path clearly. The user should not need broad home access merely to study a bundled lesson if a narrower sandbox contract suffices.

Documentation and product truth

README still describes a .NET 8 WinForms tool with 1,458 questions and four exams. Current main uses .NET 10 and has 1,598 authoring records across five banks. The interval changes a referral detail rather than those headline product facts. The audit does not act on the referral text. It identifies documentation drift that makes the project harder to assess and can mislead someone trying to install or contribute. Source-generated counts and a current supported-platform table should replace manually duplicated facts. A README build command should be executed on a clean checkout before being presented as the normal route.

The current PDF export footer says Nutanix Certification Practice. New A+ and Network+ packs must not inherit that footer accidentally. A vendor-neutral exporter should take validated track metadata and display the correct unaffiliated status, official exam code, pack version and review date. A PDF with keys and explanations is a content distribution artifact, so its eligibility must match the pack's rights status. The export should not include unreviewed or linking-only sources just because they are present in the in-memory bank. Visual verification must inspect the actual pages, including long questions and source links.

The older audit and platform plan should be retained as dated historical records, not left as current competing instructions. The new audit should clearly identify which baseline claims are superseded. A folder index can list the current audit, gap analysis and OpenSpec, with old versions labeled historical. The project convention is one top-level Drive folder per project; this package belongs in the existing RTP folder. It does not require creating a new nested project structure or widening sharing. Delivery should include actual files, usable links and word counts, with the original editable spec package preserved alongside reader formats.

Proposed acceptance evidence for every audit finding

Every finding should have an identifier, affected files, a reproducer or source locator, severity, proposed owner role, implementation task, test command and passing condition. Findings requiring expert or rights decisions need a concrete workflow and output record, not a vague "review later" label. An engineering executor can own implementing that workflow, while the authorized decision maker supplies the missing substantive choice. The

package should not pretend that a code change can grant a vendor license or decide a legal question. It can make those boundaries enforceable so unresolved material is not published or transmitted.

The traceability register should connect findings to OpenSpec requirements and acceptance scenarios. No finding should be unowned. A source-proven defect can begin with a failing test; an unverified runtime concern begins with a prescribed environment and check. A correction is complete only when the evidence demonstrates the original failure no longer occurs and adjacent behavior remains correct. A documentation-only fix is appropriate for an inaccurate label, but not for a broken boot path or false score. Where a supported feature is missing, the spec should implement it or remove the misleading claim through an explicit owner decision, with tests protecting the selected behavior.

For the event demo, the release record should identify the exact build, reviewed content pack, installed host, offline behavior, tutor state and reset procedure. A backup recording should be labeled as a backup, not live AI. The station should use synthetic data and no student accounts unless separately authorized. The presenter should be able to open the track, run the task, ask for the permitted coaching example and reset for the next group without manually deleting files. This practical rehearsal is where source fixes become dependable experience. It is also the evidence that lets Roger stop carrying every hidden failure mode himself.

Cross-platform operation, all-content demo mode and finding ownership

Roger's current direction

Roger says the platform previously worked on Windows and now needs to be cross-platform. He also says demo mode must work for all content, specifically the simulator, and that all findings must be addressed. These requirements expand the earlier two-task event slice. The package may still order work so a coherent beginner loop is built first, but the target product is not satisfied by repairing only the screens used in a ninety-second demonstration. Every existing simulator scenario needs a defined demo fixture, valid completion logic, reachable interactions and a reset path. Every audit finding needs a specified fix and acceptance evidence.

The statement that it worked on Windows should be preserved as the user's experience, not dismissed because current source is broken. The runtime audit proves that the current static module graph has a missing import and that the same import appears at the previous pinned baseline. It does not identify which executable, shell, branch, cached Web asset or older copy Roger used when it worked. An older build is one possibility, but it is a hypothesis, not a historical fact. The audit therefore says "current inspected source fails the tested module boot" rather than "the simulator never worked". Recovering the previously working build can help diagnose packaging and source drift, but should not become a blocker for fixing the current source with explicit tests.

Cross-platform also needs a precise product definition. The Web/PWA is a proposed primary common surface, with desktop shells retaining parity where approved and tested. Windows, Linux and macOS claims require real supported-host evidence. A browser route on a school Chromebook is a relevant use case, but no particular school's device or browser policy has been verified. The package should test a declared browser matrix and graceful storage/network denial, rather than assume every managed device permits service workers, local storage or external AI requests. A local offline lesson and synthetic simulator can remain useful even when online coaching is unavailable.

Demo mode is a separate operating contract

A demo mode should not simply be the ordinary learner mode with the sign-in button hidden. Its contract is repeatability and safe public presentation. It needs a clean deterministic seed, a visible mode label, synthetic data, no real infrastructure calls, a one-click reset and no requirement to retain personal learner history. A presenter should be able to start any supported scenario, demonstrate its actions and restore the initial state without manually clearing browser storage or deleting a session file. The same scenario definition should

support learner practice and demo presentation, while the session policy determines persistence and reporting. The current simulator has 44 scenarios and 78 objective predicates. Twenty-seven predicates are literal true, and sixteen scenarios are complete immediately on fresh seed. One object-bucket scenario reads the wrong collection and cannot pass. Other predicates rely on pre-existing seed counts or exclude entities by hardcoded names. These patterns are not acceptable in an all-content demo mode because they make the result dependent on hidden state or provide completion without interaction. A classroom demonstration amplifies this risk: a presenter can click “check” and see success while students have done nothing, or complete an apparently correct action and never receive completion.

The fix should define an initial fixture for each scenario, the required action trace or final state, negative paths and a repeatable completion example. A baseline fixture must not already satisfy the learning objectives unless the scenario is explicitly an observation task with a recorded interpretation response. A task such as resuming a cluster should establish a hibernated cluster in its fixture. A task such as adding a bucket should inspect `object_buckets`, not an invented collection. A task such as reading a capacity chart should require a learner response tied to the fixture’s metrics, not a count of rows that existed before the scenario began.

Demo completion should be truthful without pretending to be an official examination score. The deterministic checker can report which task conditions were met and which remain. It should not infer mastery, job qualification or certification readiness from one task. The presenter can show the evidence panel: requested state, actual state and the action that changed it. This makes the demonstration educational and debuggable. A model-generated explanation may describe the result, but cannot mark a task complete. The rules should live outside the model and produce the same outcome with AI disabled.

Scenario coverage acceptance matrix

The all-content requirement implies a matrix rather than a few happy-path screenshots. Every scenario row should identify scenario ID, track/exam mapping, initial fixture, start route, required supported actions, objective validators, expected negative result, expected positive result, reset behavior and supported hosts. Every one of the 78 current objectives needs a meaningful contract or a reviewed replacement. New A+ and Network+ scenarios add rows rather than being exempt. The matrix should be generated from the authoritative scenario manifest so the report cannot drift from the shipped catalog.

Each row needs at least three executed sequences. First, start from a clean seed and check progress before acting; it should remain incomplete where work is required. Second, perform a wrong or partial sequence; the validator should explain the missing condition without passing. Third, perform the intended sequence; only then should the task complete. Reset should restore the same fixture and make the work incomplete again. Reopening the scenario should follow the selected demo/session policy. A test that only calls a validator on prebuilt final state is useful but insufficient for the UI path: the intended screen must expose the actions that can reach that state.

The matrix should include realistic failures. A missing view import should fail startup before scenario tests begin. A route whose `afterRender` method throws should be reported as broken, not skipped because its scenario happens to pass on seed state. An async CLI handler should resolve to text, not a Promise object. A storage error in learner mode should not corrupt the current task, and demo mode should remain resettable without persistence. A transport that is unavailable should fail clearly rather than returning mocked success. A report should distinguish browser runtime, headless DOM runtime and native embedded-browser runtime, because they establish different facts.

Visual acceptance is part of this matrix. A task can emit markup in jsdom and still be unreadable on a projector or keyboard-inaccessible. The selected view should have readable instructions, current objective status, clear next action and a visible reset. Long labels and tables must fit the target browser sizes. Focus should remain predictable after dialogs and route changes. The report should preserve screenshots or recordings of decisive views and note any unverified area. A count of 43 views rendering without a JavaScript exception is not a visual readiness claim.

One simulator source, several host adapters

The duplication finding is central to cross-platform reliability. The desktop project ships the Legacy WinForms Web assets; standalone PWA uses another tree. Since October 1, the bridge clients have diverged functionally. The PWA supports WebView2/postMessage and a mock standalone response path. The desktop copy adds CefGlue and rejects when a bridge is unavailable. Neither copy is a complete superset. A blind directory overwrite could remove useful transport behavior or reintroduce false acknowledgements. The consolidation needs an explicit host adapter contract.

The shared simulator should own views, scenarios, state transitions, validators and presentation. Host adapters should own transport detection, request/reply correlation, timeouts, origin restrictions, disposal and persistence policy. Browser-only simulation should not need a desktop bridge for local state changes. Native-only operations should be feature-detected and clearly unavailable when the adapter cannot perform them. Tests should run the same scenario fixtures through every adapter without changing the expected learning result. The build should compare or generate assets so a second manually edited authoritative copy cannot appear silently.

The existing unwired Desktop/Simulators manifests should be reconciled with this design rather than left as a second catalog promise. A manifest that lists ten routes and points to absent local view files is not a working multi-vendor loader. The package should choose a current manifest schema and migrate or remove misleading stubs through an explicit task. Old paths can remain compatibility aliases if required, but there must be one source of truth for published scenario IDs and content eligibility. A test should detect an orphaned manifest route, missing view, duplicate ID and unsupported response type before build publication.

Persistence boundaries and truthful confirmations

The newer JSON session store is real, and browser StateStore can persist state. End-to-end learner resumption is not wired simply because a handler exists. The runtime verification pass found that the bundled JavaScript does not call the save/load progress handlers, and the desktop exam-session view model does not consume the simulator session store. Several bridge operations acknowledge effects they do not perform: resetting a session does not delete it, setting a preference does not change the retrieved setting, submitting an answer does not alter the reported statistics and import/export is not a faithful round trip. The fix must be a contract repair, not merely a message change.

Every effect acknowledgement should be tied to a verified state change. A save returns success only after the write succeeds. A reset removes the defined session data and confirms the new baseline. A settings update persists the allowed field and the next read returns it. An answer submission records the attempt once with stable identity and revision. Import validates a schema, previews replacement where needed and leaves existing data untouched on failure. Export includes the actual permitted data and can be imported into an equivalent clean store. Unsupported operations should say unsupported, not return hardcoded success.

Demo mode can avoid persistence requirements by design, but it cannot hide broken learner mode. The package should test both policies. Demo sessions use a deterministic ephemeral store and reset without retaining students' answers. Learner sessions use a versioned local store, have explicit save/error behavior and preserve attempts across restart. Progress should not become mixed with simulator seed state. A user who changes a scenario fixture should not overwrite learning history unless the reset scope clearly says so. These boundaries should be visible in code contracts, UI labels and tests.

Finding traceability and ownership

The following ownership roles are proposed for the implementation package. They are not assignments to named people or evidence that work has begun. The Core/scoring executor owns parser parity, response types, retry accounting, stable identity and progress migrations. The simulator executor owns boot, lifecycle, route rendering, async actions, validators, seed fixtures and reset. The desktop executor owns streak display, review export, native host disposal and bridge effects. The build/release executor owns scanner schema, coverage gates, packaging and deploy smoke. The content reviewer owns objective remapping, contradictions, source

correctness, distractors and exam-integrity evidence. The product owner supplies unresolved provider, licensing and supported-host choices through the defined review workflow.

Every role needs acceptance evidence for its findings, and the register must contain no unowned row. The requirement to address all findings does not allow a report to label unresolved rights as resolved or a native UI concern as runtime-tested. It means the package must specify how that issue is resolved, who provides the decision and what artifact proves completion. A licensing intake workflow can fail closed on missing permission; it cannot invent permission. A platform smoke can be required for macOS; it cannot be replaced by a Linux build plus a hopeful comment. The final gap and OpenSpec documents should preserve these distinctions while providing full tasks.

The minimum traceability set includes the ten content findings in the content chapter; the CI and packaging findings in the release chapter; all critical/high/medium/low Web findings in the runtime evidence; all desktop and bridge findings in the .NET evidence; and every partly open historical audit item. Overlapping findings should be cross-referenced rather than counted as independent defects. For example, question syntax appears in both content and runtime chapters, but its fix should be one canonical parser/linter parity work item. Simulator duplication and missing imports are related but distinct: consolidation alone does not repair the wrong module path.

Acceptance commands must be tied to the repository's actual project files and any newly proposed test directories. If a command is proposed rather than already executed, the document should label it. A test path that does not exist yet should be introduced by a task before its command appears in a gate. A copy-paste executor handoff should verify the base SHA, create an authorized non-main work branch only when implementation is approved, run failing fixtures, make the smallest fix and record the tested head. It should not merge, deploy or purchase as a hidden final step. The present package remains plan-stage.

Practical completion definition

The expanded feature is complete only when all selected tracks have current reviewed content structure, all simulator scenarios satisfy the demo matrix, shared assets behave consistently across the supported hosts, progress and bridge confirmations are truthful, CI fails on invalid content and security output, and released bundles load without a source checkout. The non-endorsement disclaimer must appear in the product surfaces, exports and presentation context. AI-assisted material must retain provenance and reviewer status, with no claim that a disclaimer grants rights over actual exam content. A tutor must have a bounded evidence corpus and deterministic scoring separation.

These outcomes are larger than the original short track outline. They require a sequence and measured checkpoints, but no finding is parked outside the package. The October 23 presentation can use the rehearsed subset while the full implementation plan covers all content. If the user wants every scenario implemented before that date, capacity and test results must be measured before promising it. The audit supplies current facts and acceptance boundaries; the gap analysis supplies dependency order and ownership; the OpenSpec supplies exact changes and scenarios. Together they should make implementation inspectable without requiring Roger to audit every plausible claim himself.

Web simulator / PWA audit - radicaltrainingplatform @ 904106ad388c0d48add662ddcd02153ff5f85537

Scope: the browser simulator (RadicalTrainingPlatform.Web, the PWA), the second copy under RadicalTrainingPlatform.Legacy.WinForms/LabSimulator/Web, scenario validation, and (as asked late) the NCP-AI-Part4 answer-key parser question. Read-only. I cloned the repo to a scratch directory, checked out the exact head, changed nothing in the repo, pushed nothing, and removed the one temporary file my harness created inside the checkout (git status was clean afterward).

1. Headline

The PWA cannot boot at this head. `app.js` statically imports `views/pe-reports.js`, which imports `../components/ConfirmDialog.js`. That file does not exist in either copy, on main, or on any remote branch I searched. A missing module in a static ES import graph rejects the whole graph, so `app.js` never executes, no route is registered, and the user sees the empty shell from `index.html`. I reproduced this by importing `app.js` under `jsdom: ERR_MODULE_NOT_FOUND` for `.../js/components/ConfirmDialog.js`, imported from `pe-reports.js`.

The bug is not new. The same import is present at `2b17f51` (the Oct 1 audit head), and the file history of `pe-reports.js` shows the only commit touching it is `303d089` (2026-06-11, the CertForge rename). The component was named `Confirm.js` (exports `confirm()`) and the import was never updated. Nothing in the Oct 1 to Oct 10 range touched the Web folder or the simulator JavaScript for this problem.

Everything below the headline is second order. I list it because once the import is fixed, these are what the user will hit next.

2. What changed since the Oct 1 audit (2b17f51 -> 904106a)

I diffed the two heads (49 files, +1,955/-343) and walked the log for the simulator-related paths. Relevant landed work:

- `568c232` feat(desktop): real CefBridge handlers + JSON session store. It added 47 lines to `Legacy.WinForms/LabSimulator/Web/js/core/BridgeClient.js`: CefGlue transport via `window.dotnetBridge.call` plus a `__cefBridgeDispatch` listener and queue drain. Before this the desktop bridge was inert in both directions (the file's own header comment says so).
- `8af6723`, `ff20257`, `47e35bf`, `74957e0`, `a81ed55`: Desktop fixes (stats view, Export button, Review-button NRE, Avalonia 12.1.3 package alignment). None touch the web simulator.
- Core: `SessionStore`, Errata service, `QuestionParser` changes, `HardcodedBlueprintService` routing fix (`b0d7570`). These are outside my surface except the parser question in section 7.
- No commit in the range touches `RadicalTrainingPlatform.Web`. The PWA copy is exactly as it was on Oct 1.

Net effect on duplication: the Oct 1 finding “two diverging simulator copies” is now sharper. The two copies were byte-identical except for PWA packaging; now they differ in a real functional way, because only the desktop copy got the bridge work.

3. The two simulator copies

Verified with `diff -rq` between `RadicalTrainingPlatform.Web` and `RadicalTrainingPlatform.Legacy.WinForms/LabSimulator/Web` at the head:

- Identical: all 47 route views, `BaseView.js`, `PlaceholderView.js`, all core files except `BridgeClient.js`, all components, `css`.
- Differ: `index.html` (the PWA adds the manifest link, theme-color meta and a service-worker registration block); `core/BridgeClient.js` (see below).
- PWA only: `manifest.json`, `sw.js`.

`BridgeClient.js` divergence. The PWA copy detects `WebView2`, then `window.opener/parent.postMessage`, then falls to “standalone” mode with canned mock responses (150 ms timeout, fake VM/cluster lists). The desktop copy detects `WebView2`, then CefGlue, and when no transport exists it rejects with “Bridge unavailable” and drops posts. The PWA copy also has an `off()` method and an origin check on `postMessage` that the desktop copy dropped. So neither copy is a superset. This matters for any “one simulator” plan: the shared core needs a deliberate transport layer, not a diff-merge.

Which copy the desktop app ships: `RadicalTrainingPlatform.Desktop.csproj` line 42 links `...Legacy.WinForms***` as `LabSimulatorcontent`, and `LabSimulatorView.axaml.cs` looks for `LabSimulator/Web/index.html` next to the executable. So the desktop app ships the `Legacy.WinForms` copy, and the `Desktop/Simulators` folder

(manifest.json for a “nutanix” vendor plus a `_Template`) is not wired to anything I could find. Desktop/Simulators/Nutanix/manifest.json lists 10 routes with file paths like `views/pe-dashboard.js` and references views (PeVmsView etc.) with no matching directory under Desktop/Simulators/Nutanix. It is a design stub for the multi-vendor direction, not a working loader. I did not verify the `_Template` app-loader.js at runtime.

Consequence: the pe-reports import bug is in both copies, so the desktop lab simulator fails in the same way (CefGlue loads file:// modules and a missing module has the same effect). I did not run the desktop app; this is inference from identical files plus the standard ES module behavior, and I label it as unverified at runtime.

4. “Do the 49 views actually run?”

Count. There are 49 .js files under `js/views`. Two are infrastructure: `BaseView.js` (the lifecycle base class) and `PlaceholderView.js` (a “Coming Soon” stub). `PlaceholderView` is imported nowhere (grep across `js/`). So the real number is 47 routable view modules, and `app.js` registers exactly 47 routes (grep count of `router.register`: 47; no duplicate paths; every data-route in `index.html` is registered; every import resolves except the one named above; every imported class name is exported by its module). `ServicePagesView` serves `/pe/services` as one module. The “49-view” figure from the Oct 1 audit and memory is therefore 47 views plus 2 helper files.

Runtime test. I built a jsdom harness (Node 22, jsdom, no browser) that loads the real `StateEngine`, calls `state.init()` (jsdom has no `IndexedDB`, so `StateStore` used its `localStorage` fallback; the `IndexedDB` branch is untested), then for each of the 47 view modules constructs the exported class, awaits `render()`, attaches the element, calls `afterRender()` as the Router does, and checks for empty output and for literal `undefined/NaN/[object text]`. Result: 43 of 47 render and run `afterRender` without throwing and produce non-trivial markup, with no `undefined/NaN/[object]` text and no `console.error` output. Four fail:

1. `views/pe-reports.js` - cannot be imported (missing `ConfirmDialog.js`). Severity: critical (it breaks the whole app, see section 1).
2. `views/pc-insights.js` - `afterRender()` throws “Cannot read properties of undefined (reading ‘querySelector’)” at line 84. The view calls `this.root.querySelectorAll(...)`, but nothing in `js/` ever assigns `this.root` (grep for “`this.root =`” returns nothing) and `BaseView` does not define it. Severity: high. Every visit to `/pc/insights` throws after render, so tabs never wire up and the Router’s `view.afterRender` call rejects unhandled in the `async renderView` path.
3. `views/pe-capacity.js` - identical defect at lines 100/109/164 (`this.root` undefined). Severity: high.
4. `views/ci-networking.js` - line 169 calls `state.get(‘nc2_clusters’, uuid)`, but `StateEngine` has no `get()`; the real accessor is `getById(collection, id)`. In my harness this threw during the run. Severity: high for NC2 networking: the cluster-specific networking panel cannot render. I did not trace which user action reaches line 169 first, so whether the default landing state of this view works without it is not established beyond “it threw in my harness”.

Two views (`pc-insights`, `pe-capacity`) matter beyond rendering: they back two scenarios (`mci-insights-01`, `mci-capacity-01`), which are among those that complete with no user action (section 5), so they hide behind a green result.

What I did not test: real browser behavior, layout, CSS, click-through of every button and wizard, theme toggle, hamburger menu, router redirects, service-worker lifecycle, `IndexedDB` persistence, and any visual correctness. jsdom proves “the code runs and emits markup”, not “the page looks right and the buttons do the right thing”.

5. Scenario validation and scoring

Structure (`js/views/scenarios.js`, 696 lines). 44 scenarios (21 NCM-MCI, 9 NCP-US, 7 NCP-CI, 7 NCP-AI), 78 objectives, no duplicate ids, every `startRoute` is a registered route. Each objective has a `validate()` closure that reads the live state engine.

There is no scoring. The view renders objectives with a checkbox icon and a “Check Progress” button; when every `validate()` returns truthy it shows “All Objectives Complete!” and a toast. Nothing records the result: no score, no attempt count, no timing, no history, no persistence of completion across reloads. The Oct 1 note “no

persistence” is confirmed for scenarios specifically. “Does it score correctly” therefore has the answer: it does not score at all; it checks state predicates.

I evaluated every validator against a freshly seeded state with the real StateEngine (jsdom, localStorage fallback). Findings:

A. 27 of 78 objectives are literal validate: () => true. They pass before the learner does anything. Examples: mci-04 (all three objectives), ai-03, ai-04, cli-02, cli-03, ci-04.

B. 16 of 44 scenarios are completely satisfied on a fresh seed with zero user action. From the run: mci-04, ci-04, ai-03, ai-04, cli-02, cli-03, us-05, us-analytics-01, ci-migrate-01, ci-monitor-01, ci-scale-01, ai-model-01, ai-monitor-01, ai-gpu-01, mci-capacity-01, mci-insights-01. Pressing Check Progress on any of them shows the success banner. Severity: high for a study product, since the learner sees “complete” without practicing.

C. Non-trivial validators that already pass on the seed (so even the “real” ones are weak): us-01 obj-1 (seed already has at least 3 FSVMs), us-05 obj-1 and obj-2, ci-03 obj-2 (“Resume the hibernated cluster” is satisfied when no cluster is hibernated, which is true before obj-1 is done; I confirmed ci-03 evaluates false,true on a fresh seed), us-repl-01 obj-2, ai-monitor-01 obj-2, ai-gpu-01 obj-1, mci-insights-01 obj-1 and obj-2. These check the seed data, not learner action. The state-based pattern also cannot tell “learner did it” from “was already there”.

D. us-bucket-01 can never pass. Its validator reads state.getAll(‘buckets’), a collection that does not exist in the seed (48 seed collections; the object store uses object_buckets). The only ‘buckets’ reference is that line. Severity: medium (one unwinnable scenario, a real correctness bug).

E. Hard-coded seed names: ci-01 and ci-02 exclude clusters by seed name (NC2-Prod-AWS, NC2-DR-Azure) and ai-01 excludes GPU-Node1-Slot1. If the seed is renamed or reset to different data, these pass or fail wrongly. Several validators use count thresholds on seed size (applications > 2, lcm_update_history > 2, floating_ips > 2, alert_policies > 2): they depend on the seed having exactly two rows, so any change to seed counts breaks them. Severity: low to medium, a maintenance trap.

F. Initial render error handling: #renderActiveScenario wraps the whole results.forEach in one try/catch, so a throwing validator skips every objective after it on first render (they all show unticked). Check Progress handles each objective separately. Severity: low.

G. Reset interaction. state.reset() and importState() change the data under a scenario; scenarios do not reset or snapshot state on Start, so a learner who completed a scenario earlier starts the next one with stale entities (for example Lab-WebServer exists). Not run end to end; deduced from code (startScenario does not touch state). Severity: low.

Positive check: I scripted the realistic completion for mci-01 through the StateEngine API (create VM with disk image, NIC subnet, power on) and all four validators returned true. Through the CLI service I ran the hint commands for cli-01 (ncli container create name=CLI-Container rf=2, acl net.create name=CLI-Net vlan=500): both mutated state, and the validators would pass. So where validators are real, they work. I did not script-complete the other 30-odd scenarios, so their completion path is untested.

6. Other runtime defects found

1. CLI “alert” command returns a Promise. CLIService.#alertCmd is declared async (line 665) while execute() returns the handler’s value synchronously, so execute(‘alert list’) returned an unresolved Promise (serialised as {} in my run). CLITerminal.js line 97 does const result = this.#cli.execute(cmd) with no await, then appendOutput(result) uses textContent, so the terminal prints “[object Promise]” for alert list and alert resolve. This directly breaks cli-03’s hint (“Run alert list”), although that objective is literal-true so nobody notices. Severity: medium. Verified for the CLIService return value; I did not drive the terminal UI, so the “[object Promise]” display is derived from the two code lines.
2. Audit log write ordering. StateEngine.create/update/remove call #persist() first and #logAudit() after, and #logAudit pushes to state with no persist. In the harness, after state.create(‘vms’, ...) the live audit_log had 1 entry while the persisted localStorage copy had 0. Audit entries therefore reach storage only on the

next write, and the last action before closing the tab is lost. It also affects ops-01 on reload. Severity: low to medium.

3. Router has no guard against overlapping renders. `#renderView` is async and sets `#currentView` before awaiting `render()`; two quick hashchanges can interleave and append two views into the container. Not exercised in a browser. Severity: low, flagged as code-reading only.
4. Router fallback. An unknown hash navigates to `/pe/dashboard`, which is fine; the breadcrumb builder in `app.js` capitalises path segments naively ("`Nc2-console`" style labels). Cosmetic.
5. Service worker. `sw.js` precaches 62 explicit asset paths (all exist) under a fixed cache name `labsim-v1`. It does not include `ConfirmDialog.js`, consistent with the file not existing. A fixed cache name with a hand-maintained asset list risks stale update behavior. Its exact effect requires a real service-worker update test; generate/version the asset manifest and verify returning-user updates rather than assuming a cache-name change is the only browser update mechanism. I did not run the service worker. Severity: medium for the fix rollout (it will matter the moment the import is repaired).
6. Deploy. `.github/workflows/pwa-deploy.yml` publishes `RadicalTrainingPlatform.Web/` to GitHub Pages on any push touching that path, with no test or import-resolution step. That is how a broken static import stayed deployable since June. I did not check whether the live Pages site is currently serving a broken build (not fetched); main has been in this state since June.

7. NCP-AI-Part4 "B and D" answer keys: confirmed

The static finding is correct.

- NCP-AI-Part4.md lines 805, 819, 833, 847, 861, 875, 889, 903, 917 and 931 hold the keys for Q61 through Q70, written as "Answer: B and D", "B and E", "B and C", "A and B" and so on. Verified with `awk` over the file: exactly these ten answer lines fail the strict form, and no other exam file has any answer line outside the comma/space/concatenated forms (NCM-MCI-Part3 and NCP-CI-Part4 and NCA-75 parts are clean).
- `Core/Services/QuestionParser.cs` line 44: `AnswerRegex` is `^(?:Correct )?Answer:([A-F][,](?:[A-F][,])*)**` with `RegexOptions.Compiled` only (no `IgnoreCase`). After the first letter the group allows only commas and whitespace before another A-F, then it requires the closing `****`. The word "and" breaks it (lowercase a is not in A-F). I evaluated the identical pattern in Node: "Answer: B and D" returns no match; "Answer: B, D", "Answer: BD" and "Answer: B D" match. I did not run .NET because dotnet is not installed in my environment; the pattern semantics used here are identical in JS and .NET, but this is the one step not run in the real runtime.
- Effect in the parser (`QuestionParser.cs` lines 253-262 and 290-293): the answer line does not match, so `CorrectAnswers` stays empty, the "Answer..." line gets swallowed into the explanation text, and the question is dropped by the "`Options.Count > 0 && CorrectAnswers.Count > 0`" check, logging a warning "Skipped Q{id}... Answers=0". So ten NCP-AI questions silently disappear from any exam built from this parser. No crash.
- `scripts/lint-content.py` line 21: `VALID_ANSWER` is `^(?:Correct )?Answer:` with no closing anchor, so it accepts "B and D" and the lint passes. Lint and parser disagree, which is exactly how this survived. There is no errata entry for NCP-AI-Part4 (`errata.json` has none), so the Errata override cannot rescue it either.
- Fix options for whoever implements: normalise the ten lines to "B, D" (content fix, matches lint and parser), or extend the regex to accept "and"/"&" and tighten the lint to the same pattern so both tools agree. Either way, add a CI check that parses every file with the real parser and compares question counts to the lint count.

8. Severity summary

Critical - C1. PWA (and by inference desktop simulator) does not boot: `pe-reports.js` imports non-existent `components/ConfirmDialog.js`. Both copies. Fix must use the actual component export and relative path; from `views/pe-reports.js` the component path is `../components/Confirm.js`. Add an import-graph regression, not only a rename. Verified (`jsdom` import of `app.js`).

High - H1. pc-insights.js and pe-capacity.js use this.root, which nothing sets; afterRender throws. Verified. - H2. ci-networking.js calls nonexistent state.get(); use getById. Verified. - H3. 16 of 44 scenarios complete with no action, 27 of 78 objectives are literal-true, and more pass on seed data. No scoring or persistence. Verified. - H4. NCP-AI-Part4 Q61-70 are dropped by the parser while lint passes. Verified by regex evaluation; .NET run not done.

Medium - M1. us-bucket-01 unwinnable (validator reads missing 'buckets' collection). - M2. CLI alert command async/Promise mismatch (alert list prints nothing useful). - M3. Audit entries persisted one write late. - M4. Service worker fixed cache name/manual asset list: update behavior needs a real-browser regression and generated version policy. - M5. Two simulator copies, now functionally diverged in BridgeClient.js; desktop ships the Legacy.WinForms copy; Desktop/Simulators is an unwired stub. - M6. Pages deploy has no import or smoke check.

Low - L1. Hard-coded seed names and counts in validators. L2. Whole-list try/catch on initial scenario render. L3. Router overlapping renders. L4. Scenarios do not reset state on start. L5. PlaceholderView unused; breadcrumb label cosmetics.

9. Tested versus unverified

Verified by running code (jsdom, Node 22, real project modules): app.js import failure; per-view render and afterRender for all 47 modules; scenario validators evaluated on fresh seed; completion of mci-01 via state API; two CLI hint commands; CLI alert return type; audit persistence gap; regex behavior for the Part4 answer lines.

Verified by reading code and diffs only: router race, scenario state carry-over, desktop shipping path, service worker staleness, Pages workflow, CefGlue behavior, Desktop/Simulators stub status, that the desktop simulator is broken by the same import.

Not verified at all: any real browser or CefGlue run, IndexedDB path, visual layout and CSS, every button/wizard flow in all 47 views, the remaining scenarios' completion paths, .NET execution of the parser and test suite, whether the deployed Pages site is live and what it serves, the legal/rights items from Oct 1.

Branches: I searched all 16 remote branches (including chore/external-audit and fix/vuln-gate-blocking-2026-10-03) for components/ConfirmDialog.js and found none, so I call it missing rather than stranded. I did not inspect each branch's simulator files for other differences.

.NET audit: build health, desktop flows, bridge behavior, and persistence

Audit date: October 10, 2026. Reviewed head: 904106ad388c0d48add662ddcd02153ff5f85537. Comparison baseline: 2b17f51, the October 1 audit head. Scope: Core, the Avalonia desktop application, CEF bridge, new session store, related tests, and the .NET vulnerability gate. No tracked source was changed, no commit was created, and no branch or PR was pushed.

Executive assessment

The .NET application has materially improved since October 1. The current Core suite passes independently, Desktop builds without warnings, the new Desktop suite passes, and the exact-head GitHub Build run succeeded on both Linux and Windows. Several earlier dead paths now perform real work: review no longer dereferences a cleared session, Stats accepts real session data, Blueprint receives service data, Export reaches QuestPDF, bridge catalog/question handlers use Core, errata is applied during parsing, and the new session store writes and reloads progress. Those fixes should not be repeated as unresolved defects.

The distinction is between working components and complete learner workflows. The new persistence component does not make desktop study sessions resumable. The bundled simulator has its own browser-state store and no normal call site for the new host save/load handlers. Several bridge operations still report success without performing their named effect. Ordering questions are graded as unordered selections, repeated

attempts can put one question into both correct and wrong sets, and ordinary navigation discards the live study-session reference. The vulnerability gate also has a reproduced fail-open parser bug: real NuGet scanner output containing a High advisory produces a passing gate.

Recommended verdict: buildable and materially improved, but not release-ready on the strength of passing test counts alone. The highest priority corrections are scanner-result enforcement and grading semantics. False bridge acknowledgements, persistence integration, question identity, and simulator lifetime should follow before expanding the feature claims. This section is a functional/code audit, not a screenshot-based design sign-off.

1. Method, commands, and evidence boundaries

A live git ls-remote <https://github.com/TheArchitectit/radicaltrainingplatform.git> check resolved both HEAD and main to the audited SHA. A fresh clone was detached at that commit. Targeted line-numbered reads, diffs against 2b17f51, runtime tests, scratch counterexample projects, and remote-ref searches established the state. Plans and comments were treated as descriptions to test, not proof that the implementation matched them.

The clone fetched all advertised remote branches. The checked set was main, chore/external-audit, fix/vuln-gate-blocking-2026-10-03, and thirteen Dependabot branches spanning checkout, setup-dotnet, Core/Test packages, QuestPDF, CefGlue, and Desktop package groups. Searches across those refs found no ISessionStore usage in MainWindow or ExamSessionViewModel. Targeted comparisons checked gate, streak, cached simulator, bridge, and store files. The fix branch carries the same scanner-parser bug. Older refs either lack the gate or contain older logic. The incorrect streak and cached-lab patterns remain in inspected older refs. This is a named-ref search, not a claim that all unreachable commits or other repositories were exhaustively inspected.

An isolated .NET SDK 10.0.401 with runtime 10.0.12 was installed outside the repository. The first direct Desktop-test executable launch could not find that temporary runtime. Setting DOTNET_ROOT corrected the audit environment, and the suite passed. That initial environment setup issue is not counted as a product defect. Scratch projects remained outside the repository and referenced production project assemblies. They were counterexample harnesses, not proposed source edits.

Check	Command/mechanism	Result
Head identity	git ls-remote against live remote	HEAD/main matched full audited SHA
Core tests	dotnet test RadicalTrainingPlatform.Core. Tests/RadicalTrainingPlatform. Core.Tests.csproj --configuration Release	129 passed, 0 failed, 0 skipped
Desktop build	dotnet build RadicalTrainingPlatform.Deskt op.Tests/RadicalTrainingPlatfor m.Desktop.Tests.csproj --configuration Release	0 warnings, 0 errors
Desktop tests	Built Desktop.Tests executable with DOTNET_ROOT set	13 total, 0 errors, 0 failed, 0 skipped

Check	Command/mechanism	Result
Extra headless probes	Scratch xUnit/Avalonia project using production Desktop	2 passed: incorrect streak and null review export reproduced
Core/bridge probes	Production parser, VM, store, and CefBridge.Call in scratch executable	Ordering, retry, false bridge effects, and duplicate IDs reproduced
Current package scans	dotnet list package --vulnerable --include-transitive --format json for Core/Core.Tests/Desktop	No vulnerability entries in current results
Gate counterexample	Same scanner and unchanged gate on scratch Newtonsoft.Json 12.0.1 project	High advisory ignored; gate exited 0
Native simulator	Headless click probe	CEF exited for missing X server/\$DISPLAY; native revisit not verified

The live Actions API returned an exact-head successful Build run:

<https://github.com/TheArchitectit/radicaltrainingplatform/actions/runs/37335624087> . Windows job:

<https://github.com/TheArchitectit/radicaltrainingplatform/actions/runs/37335624087/job/111849537046> . Linux

job: <https://github.com/TheArchitectit/radicaltrainingplatform/actions/runs/37335624087/job/111849537380> . Both

jobs and their listed steps succeeded. CI status was therefore checked live, not remembered. Job logs and numerical coverage reports were not independently inspected.

Evidence is labeled carefully below. Repository tests and builds ran against the named head. Scratch probes exercised actual production classes or actual scanner output. Static findings trace deterministic source paths but were not reproduced in a full native session. That distinction lets a passing suite coexist honestly with uncovered defects. No native CEF acceptance run, independent Windows execution, packaged-install test, screenshot review, or complete PDF layout review is claimed.

2. Fixes that landed after October 1

Dependency alignment and a real desktop test layer

Commit a81ed55 aligns Avalonia, Avalonia.Desktop, Avalonia.Themes.Simple, and Avalonia.Fonts.Inter on 12.1.3 in Desktop.csproj. This addresses theme/XAML runtime mismatch risk, which compilation alone may not catch. The current Desktop build and real App headless tests passed, so the alignment has been exercised. It is appropriate to close the earlier mixed-version concern for this head while still reserving native/published-platform acceptance.

Commit 9a59e86 adds Desktop.Tests and runs it in the Linux workflow. The project uses xUnit v3 and Avalonia's headless adapter. TestAppBuilder configures the real Desktop App, not a fake shell. MainWindowSmokeTests constructs the window, triggers load/layout, clicks routed sidebar events, and verifies that a real QuestionView has a real session. This catches composition, resource, XAML, and handler failures that Core tests cannot see.

All thirteen tests passed here. Six are window smoke tests and seven are bridge tests. They do not inspect screenshots, open native CEF, simulate all pointer/keyboard behavior, or launch a clean installed bundle. The

correct conclusion is that a useful new layer exists and passes, not that the entire application has thirteen acceptance tests covering every feature. The later findings are mostly at cross-component boundaries the suite does not assert.

Review repair and exception diagnostics

Commit 74957e0 repairs the old Review-button null-reference path. `MainWindow.axaml.cs:278-319` captures the study VM before switching, creates a review VM, retains the original, and restores it through Back to Study. Repository tests cover entering review with a wrong answer and returning with the original wrong count intact. Both passed independently. The guaranteed old crash is fixed at this head.

`App.axaml.cs:42-58` adds last-resort exception logging. UI exceptions are logged and marked handled; background/unobserved task exceptions are logged. This is better than an unexplained process exit. It does not restore a failed operation, and the comment recognizes that. Credit the diagnostic floor without describing it as correctness recovery. A failed export/save can still need a visible error or retry path.

Stats, blueprint, export, and live question presentation

Commit ff20257 makes Stats reachable, removes sample numbers, and adds streak/domain metrics to the VM. `StatsView.axaml.cs:26-48` shows an honest empty state and otherwise derives values from the supplied session. Its headless route test passes. The remaining sidebar streak bug is a different output path and should not erase the improvement to Stats itself.

`MainWindow.axaml.cs:240-264` now asks the blueprint service for the relevant exam, calculates coverage from question text, and feeds `BlueprintView`. Commit b0d7570 corrects NCM-MCI routing/static initialization; 3539db2 adds pipeline testing. The passing smoke test establishes data plumbing and a no-throw route. It does not certify every objective's factual mapping or canvas layout.

Commit 47e35bf wires Export to `GenerateStudyGuidePdf` and `QuestPDF`. The existing test receives nonempty bytes starting with %PDF. That test passes in ordinary study mode. Export is no longer a dead TODO, though review mode still exposes a canonical-identity bug described later. Nonempty bytes and magic headers are not a visual pagination check.

Commit 8af6723 makes the question domain badge live, removes dead code, and fixes the releases URL. `QuestionView.axaml.cs:113-115` uses `CurrentQuestion.Domain` rather than a fixed GENERAL label. Those are real improvements. A correct URL does not establish a downloadable release exists; the live releases API returned zero releases at review time.

Shared composition, discovery, errata, and encoding

Commit 526ad86 extracts `AddRadicalTrainingPlatformCore`. `ServiceCollectionExtensions.cs` registers filesystem, exam repository, errata, parser, blueprint, reference, and session store centrally. Desktop and tests use it. Current tests and probes resolve those services correctly. This reduces duplicated composition drift across hosts, but it does not automatically wire a service into every consumer.

Commit 93ea0c2 repairs catalog counts and excludes non-exam markdown.

`MarkdownExamRepository.cs:118-138` verifies real question headers after filename filtering. That prevents project markdown becoming zero-question exams merely because it has a .md suffix. Catalog and total bridge values now derive from Core instead of stale literal rows. The production parser in this review returned 1,588 questions across five exam codes. This is a parser observation, not a certification that all source questions are correct or that every script counts the same population.

Commit 5df36fc applies answer-key errata. `QuestionParser.cs:276-287` calls the provider after a valid parse and substitutes corrected keys. `JsonErrataStore` uses source filename plus local question ID, correctly acknowledging that numbering restarts across part files. It supports `answerKey` corrections, not arbitrary option/stem edits. Missing/malformed JSON follows the declared nonfatal behavior; unsupported correction

types are logged/skipped. The current errata tests belong to the passing 129-test Core suite.

Commit 09106e1 adds RegisterSearchPath. Registered directories take priority, and registration invalidates cached paths. The scratch headless harness used that seam when its runner's working directory differed from the checkout. That required no production edit. It also illustrates why content discovery inside a checkout cannot alone prove installed-bundle discovery works.

Commits 9573b52, 720ad00, and 904106a add BOM/encoding fixes and fixtures. QuestionParser.cs:138-146 removes U+FEFF and normalizes CRLF/CR before parsing. The encoding fixtures pass in the full Core suite. Credit that exact tested behavior, rather than claiming support for every encoding or transport seam. Candidate discovery happens before parsing and has its own regex; parser normalization does not establish identical normalization at all earlier stages.

Real store and real bridge reads

Commit 568c232 introduces ISessionStore/JsonSessionStore and replaces several fake CefBridge handlers. The store supports save/load/has/delete/list under app data with instance locking. Core composition registers it. The bridge uses injected parser/store services and derives catalog, question data, and total bank size from Core. Save/load no longer merely acknowledge progress without writing it.

Store tests cover exact text round-trip, overwrite, deletion, listing, empty state, traversal rejection, and directory creation. The real bridge test drives Call(json), captures the script envelope, and saves/loads a unique session through the actual store. All pass. That closes the component implementation concern. It does not close desktop study resumption, browser-host state unification, every effect handler, or atomic durability.

3. NET-01, High: the blocking vulnerability gate fails open

The new gate is invoked as a blocking workflow step. It handles scanner failure and malformed JSON by setting failure, and allowlist entries have expiry checks. Those design intentions are sound. The actual JSON consumer, however, cannot read the output produced by its own scanner command. At `.github/scripts/vuln_gate.py:72-73`, the loop accepts only a root list and looks for direct packages arrays. The real scanner emits an object with projects, frameworks, and topLevelPackages/transitivePackages.

This was verified with the real scanner, not only a handmade fixture. An isolated net10.0 project outside the repository referenced Newtonsoft.Json 12.0.1. After restore, dotnet list `/tmp/deep-research/rtp-dotnet/scanner-probe/probe.csproj package --vulnerable --include-transitive --format json` reported a top-level Newtonsoft.Json package with High severity and advisory `https://github.com/advisories/GHSA-5crp-9r3c-p9vr`. The root was an object. The test dependency was not added to RTP.

Running the unchanged repository gate on that project printed OK/no vulnerable packages and VULN GATE: PASS, exiting zero. The root object was silently skipped, leaving hits at zero. This directly disproves the blocking guarantee. It is High severity because CI can admit an advisory-bearing package while asserting that vulnerability enforcement passed. Green CI still proves builds/tests succeeded, but cannot certify this detector works.

There is also a field-name mismatch. Actual scanner output uses advisoryurl; line 82 reads advisoryUrl. Fixing root traversal alone would still lose advisory evidence unless corrected. Traverse every project/framework and both package collections; fail closed on unsupported schema instead of converting it to clean. Add a real-shaped High fixture, transitive findings, multiple frameworks/projects, clean result, expiry/revocation cases, malformed data, and scanner failure tests.

Current Core, Core.Tests, and Desktop scans each returned no vulnerability entries. This audit is not alleging that those current dependencies are vulnerable. The counterexample establishes a broken enforcement mechanism, not an invented current package incident. The workflow's gate list also omits Desktop.Tests and Legacy WinForms; broaden coverage after parser correctness, because adding projects cannot help a consumer that ignores all root objects.

4. NET-02, High: ordering is graded as unordered multi-select

NCA-75-Part3-GapFill Q17 explicitly asks for an LCM sequence and gives D,C,B,A. Q16/Q18/Q19 also require order. Models/Question.cs:12 treats all multiple-letter keys as multi-select. ExamSessionViewModel.cs:16 stores selected answers in HashSet. IsCorrect at :63-64 and Submit at :104 sort both sides before equality. There is no question kind preserving sequence semantics.

A runtime probe parsed the real file with production QuestionParser, selected Q17, and constructed the production VM. Expected key printed D,C,B,A. The probe selected A,B,C,D. Submit returned True and IsCorrect was True. The reverse sequence was accepted because comparison erased order. This is not a hypothetical interpretation or only a misleading label; it is a reproduced wrong grade on actual content.

For sequence items whose answer includes all offered letters, selecting everything can satisfy the set equality without demonstrating order. The educational contract is lost even when explanation text is correct. This can teach an incorrect operational sequence as correct, warranting High severity. More vendors/content would spread the same failure until the model distinguishes ordered and unordered responses.

Introduce an explicit kind, preserve ordered user input, and compare lists for ordering. Genuine multi-select must retain set comparison. Parse the type from an explicit source/schema field. Add an ordering interaction rather than toggle cards alone. Tests should include correct/reverse sequences containing identical letters, partial sequences, duplicates, and multi-select chosen in another order. Until implemented, excluding these items from automatic scoring is safer than reporting false correctness.

5. NET-03, Medium: retry outcome sets can contradict each other

Submit adds a question key to the correct or wrong set but never removes it from the opposite set. Navigation/JumpTo calls ResetState, clearing submitted/selection state so the question can be answered again. The relevant paths are ExamSessionViewModel.cs:99-126, :129-147, and :182-194. Keys already include source identity, so this is not the separate bridge ID collision.

The scratch probe created a one-question session from parsed content, answered correctly, jumped to index zero, then answered incorrectly. State became CorrectCount=1, WrongCount=1, Total=1. One unique question now contributes mutually exclusive outcomes. Wrong-then-correct similarly retains the wrong key and can keep a corrected item in wrong-answer review. Domain figures and completion feedback inherit that ambiguous policy.

Decide whether grade means first attempt, latest attempt, or all attempts. First attempt should retain an immutable initial result with later practice separate. Latest attempt should replace prior outcome. All attempts require attempt records, not sets of unique question keys pretending to count attempts. Tests must cover both retry directions, skips, revisiting without submission, review membership, and the invariant that unique answered items do not exceed total questions.

6. NET-04, Medium: bridge effect replies still lie

Catalog, question reads, save/load, and total bank size are real. Neighboring effect handlers remain stubs. CefBridge.cs:289-299 logs submit_answer then returns recorded=true. :334-336 always returns inactive session. :338-342 logs reset_session and returns reset=true without deletion. :345-349 exports empty CSV. :351-356 counts newline splits as imported rows. :369-376 logs set_setting and returns ok=true; get_settings always returns defaults.

Production Call/envelope probes establish each effect mismatch. A real store saved answered=7/correct=5, but get_stats returned zeros. reset_session returned reset=true, while store.Has stayed True. set_setting(showTimer=false) returned ok=true, then get_settings returned showTimer=true. submit_answer returned recorded=true and stats remained unchanged. Importing the literal bad data returned imported=1, then export_results returned empty CSV and rowCount=0.

These are false-success contracts, not merely private TODOs. A consumer can tell a learner an effect

completed or clear local state based on host acknowledgement. Unsupported effects should return explicit unsupported responses until implemented. The normal bundled JS does not currently reach most of them, limiting present UI exposure but not making the API truthful.

The `get_stats` comment at :247-249 claims answered/correct come from persisted sessions. Lines :254-257 instead hardcode all learner fields. Only `totalQuestions` is live. The repository test checks `totalQuestions` exceeds 1,000, so it passes despite fake history fields. Test persisted stats after writes, reset deletion, settings readback, import validation, and actual export content. Do not report S22/no-stub-data broadly closed while these operations remain success-shaped no-ops.

7. NET-05, Medium: real storage is not desktop study resumption

The store has positive properties worth retaining. Session IDs containing slash, backslash, colon, or period are rejected. Save/Load/Has/Delete/List lock through the same object. Raw progress JSON is wrapped as a JSON string and unwrapped, preserving text exactly. List sorts IDs, and the default provider writes under application data instead of source directories. Tests and the real bridge round-trip prove those component operations work. Describing session persistence as wholly absent would now be wrong.

The missing connection is from actual study state to the new store. `ExamSessionViewModel` receives questions, code, and optional limit. It has no injected store, persistence schema, snapshot method, or restore constructor. `MainWindow` creates new VMs and never saves them. Targeted searches found no `ISessionStore` reference in these classes on main or advertised branches. Therefore closing the desktop app does not acquire study resumption simply because `ISessionStore` is registered in DI. A service registration is capability plumbing, not a user workflow.

Browser simulator persistence is another system. Its `StateStore.js` uses `IndexedDB` with `localStorage` fallback. Searching the bundled Web tree found `bridge.post('ready')` in `app.js` and no normal `save_progress/load_progress` call. The host handlers are reachable in tests or from a future caller, not presently used to save browser state through the usual UI. This review does not claim browser persistence is absent; it distinguishes it from the .NET session files. `IndexedDB` may preserve simulator entities while the study VM remains memory-only and host session files remain unused.

A completion plan should define a versioned session schema, stable question identity, question index, selected answers, scoring policy, optional attempts, and timestamps. Separate snapshot/save/restore responsibilities from visible views. Add autosave at deliberate points and a restored-session UI. Define how reset affects disk history and browser state. Test closing/reopening, switching modes, moving through Stats/Blueprint, version changes, and missing/corrupt files. That is the acceptance scope required before claiming a learner can stop carrying their study progress.

It is useful to close two tickets separately: storage engine implemented versus desktop resumption integrated. Combining them into one checkbox hides the real progress and the real gap. The bridge tests are good unit/integration evidence for the engine but are not an end-user resume test. A browser datastore and a host datastore can each round-trip perfectly while no shared definition of a session connects them.

NET-12: durability hardening remains untested

`SessionStore.cs:48-67` writes directly to the destination file, then deserializes its content on load. `DefaultFileProvider` delegates to `File.WriteAllText`. There is no temporary-file replacement, checksum/version envelope, backup recovery, or corrupted-file handling. Same-instance locks prevent concurrent operations in that store but do not make an overwrite atomic across process interruption and do not coordinate multiple application processes.

This is a source-supported hardening risk, not a reproduced data-loss incident. No disk failure, kill-during-write, truncated-file recovery, or multi-process test was run. The right recommendation is to implement atomic replacement and safe recovery before promising durability, then fault-test it. Do not report that a specific user's progress has been lost. A tested normal round-trip answers whether the code can write/read healthy data, not

whether it preserves old valid data after a failed overwrite.

IDs are also validated by a small denylist rather than a platform-complete allowed format. Current traversal checks are useful, but Windows reserved names and other invalid filename characters can still reach filesystem errors. This was not separately reproduced on Windows and should remain a hardening note. A constrained ID scheme such as internally generated opaque IDs would reduce that cross-platform surface. Avoid expanding the finding into an unsupported claim of arbitrary filesystem access; path separators and traversal periods are currently rejected.

8. NET-06, Medium: normal sidebar navigation drops the study VM

MainWindow.ReleaseCurrentView clears `_session` at line 129. Stats and Blueprint call it. Stats takes a snapshot first, allowing the panel to show the figures as they existed at click time, but no ordinary Back to Study path retains that VM. Blueprint keeps the last exam code for context, not the complete session. Clicking the exam again reaches `StartSession` at :140-155 and creates a fresh VM. The new disk store is not integrated to repair this loss.

This is a deterministic source path, not separately demonstrated by a complete sidebar-return probe in this audit. The finding is loss of the current in-memory continuation through a normal inspection detour. It is not evidence of deleting a saved session, because that study session was not saved. Review has explicit retention/resumption, which shows a local working pattern for navigation that the other views lack.

The distinction between navigation, reset, and new session should be deliberate. A learner commonly opens Stats to inspect the session, not reset it. An explicit Reset button already exists. Keep study state independent from MainContent.Content, and clear it only for defined reset/new-session actions. Tests should answer a question, record the current index, open Stats/Blueprint, return, and compare identity, answers, counts, and current question. Tests that only assert a panel exists miss the state loss.

The subscription ownership should follow that separation too. ReleaseCurrentView does not unsubscribe MainWindow's session PropertyChanged handler before clearing its reference, unlike ShowReviewSession. When a retained view or another owner keeps the VM alive, stale callbacks can refer to a different active session. No memory-leak benchmark or harmful stale-callback scenario was reproduced here, so this is an implementation cleanup concern rather than another counted confirmed defect. A session controller with clear subscriptions would make both continuation and lifecycle easier to reason about.

9. NET-07, Medium: disposed simulator instance is cached for reuse

MainWindow.axaml.cs:127-128 disposes a lab control when navigating away. It does not clear `_labView`. The next simulator click reaches :364, whose null-coalescing assignment reuses that nonnull object. LabSimulatorView.OnLoaded at :54 refuses startup when `_disposed` is true. Dispose permanently sets that flag and nulls the bridge/browser. The application therefore treats a disposed object as a reusable view.

LabSimulatorView comments correctly explain that visual-tree removal should not shut down a process-wide CEF runtime. Removing its Unloaded-to-Dispose subscription was meant to avoid that problem, but MainWindow still disposes the control on the same navigation event. The two classes' lifetime decisions disagree. Preventing duplicate dispose does not restore a browser after the cached object has been disposed once.

A headless UI probe attempted native opening. CEF loaded far enough to report native startup warnings, then exited because no X server/\$DISPLAY was available. Thus this review does not claim a native second-visit blank screen was observed. The source path is deterministic, while native revisit and shutdown acceptance remain unverified. The passing thirteen-test repository suite does not exercise this opening path and cannot certify it.

Choose one ownership policy. Either keep the live control through navigation without disposing it, or dispose it and clear the cache so the next visit creates a new control. Coordinate shutdown explicitly. MainWindow lacks a window-close lab-disposal hook in the inspected source. Do not insert speculative direct

CefRuntime.Shutdown/Initialize calls without checking the wrapper package's ownership. Absence of an explicit Initialize call is not itself reported as a bug, because CefGlue wrappers can initialize internally.

The native test failure is also a warning against describing the fallback as universally noncrashing. Managed catch blocks cannot necessarily intercept a native library terminating the process. This was an audit-environment missing-display case, not an end-user supported platform failure. A native acceptance environment should test first load, missing runtime, failed startup, tab leave/return, and window close with supported runtime/display conditions.

10. NET-08, Medium: EXAM:number cannot uniquely address current questions

The real get_question handler documents keys like NCA-75:12 and returns the first numeric match.

CefBridge.cs:302-320 performs that lookup. QuestionParser.cs:193-198 preserves each file's number and SourceFile. LoadAllExams at :309-321 concatenates parts without renumbering. Part files restart Q1, so the bridge's key is ambiguous even though the VM and errata correctly include SourceFile in their own identities.

Runtime inventory establishes current collisions. NCA-75 has 140 parsed questions and 40 repeated numeric IDs; its Q1 appears in Part2, Part3-GapFill, and Part1. NCM-MCI has 360 questions and 80 repeated numeric IDs. NCP-US has 400/80, NCP-AI 325/80, and NCP-CI 363/80. A caller cannot ask for each source's Q1 via one EXAM:number namespace. It receives whichever match enumeration placed first.

The existing bridge test only asks NCA-75:1 and asserts it returns a stem/options. It passes even when several distinct questions satisfy that key. This is the important tested-versus-complete distinction: the read is real but identity is under-specified. Using this API for answer history, cross-links, imports, or review would silently attach data to the wrong item or leave later part items unreachable.

Use an immutable source-aware ID or a stable bank-wide mapping, and keep display ordinals separate. Tests should request two Q1 items from two files in one exam and assert their distinct stems. Existing consumers need compatibility/versioning if they already persist EXAM:number. Changing enumeration order is not an identity fix; it merely changes which question wins the collision. The Core model already carries enough source information to design the corrected key.

11. NET-09/10: two UI edge defects reproduced headlessly

The sidebar streak still formats CorrectCount at MainWindow.axaml.cs:179, despite the newly implemented VM.Streak. A production headless probe answered correctly then incorrectly. VM.Streak was zero, but the actual TxtStreak TextBlock contained 1 with the fire icon. Both test assertions passed. StatsView's BestStreak use is correct; this is the remaining main-window output path, not absence of Core streak support.

UpdateStats also responds only to CorrectCount, WrongCount, and AccuracyPercent changes. When correcting the text property, include Streak notifications, particularly for Skip, which resets streak without following every normal submission notification path. This is Low severity motivational feedback, but it makes a claim of fully genuine streak display premature. A small test that inspects the actual TextBlock after a miss closes it directly.

Review export is also confirmed. Review creates a VM whose ExamCode is CODE (Review), using a display label as canonical identity. GenerateStudyGuidePdf chooses that current code before _lastExamCode, fails the bank dictionary lookup, and returns null. The headless probe got real PDF bytes before review and null in review. This is a Low/Medium feature edge case, not the old dead Export button.

Separate canonical exam identity from mode/label. The same identity should serve export, blueprint, reset, lookup, and persistence. Otherwise each new consumer must understand and strip presentation text, creating more silent failures. Review can display its mode without changing the bank ID. A regression test should enter review and still generate the intended study guide, or explicitly show a supported review-specific export if that is the product choice.

No visual rendering claim follows from these headless assertions. They inspect actual control values and production event behavior, not pixels. Screenshots should verify layout, clipping, labels, and readability separately. Similarly, PDF bytes prove generation reaches QuestPDF, not that the page looks correct. This

report intentionally does not treat nonempty output as visual verification.

12. NET-11: build output is not a tested end-user bundle

Desktop.csproj copies the Web tree from Legacy WinForms into LabSimulator/Web. The generated Desktop output contained 65 files there, supporting the deployed HTML path. It did not contain top-level exam markdown or errata.json. The project has no corresponding content-copy declaration for those banks. Tests find them from the checkout through working-directory/assembly-parent searches; a clean installed directory cannot depend on that source tree.

This is a readiness gap, not a claim that all launches are empty. A user could manually put content in searched app-data paths, and development launch works in the checkout. A release needs to state its supported content route and test it. Copy only the intended distributable to a clean directory, launch with a neutral cwd, discover supported banks, and confirm errata. Ensure the test cannot accidentally walk up into repository source.

The live GitHub releases API returned an empty list. LabSimulatorView.cs:130-132 and multiple failure messages tell users to download a desktop bundle from releases; the download handler uses the actual repository page. That URL is correct, but no published release was listed at audit time. Successful CI artifacts are a different distribution surface. Do not claim no artifacts exist, but do not promise a readily available release bundle without confirming it.

The Linux workflow appropriately avoids the Windows-targeted solution and builds Core/Desktop individually. Windows restores/builds the solution, runs Core tests, and uploads WinForms output. Both exact-head jobs succeeded. WinForms was not independently run here on Windows. Native WebView2, OS dialogs, and installed Windows behavior remain CI/source-reviewed only. A successful solution build proves compatibility enough to compile, not feature parity.

Legacy LabSimulatorBridge registers ready and log, a much narrower handler surface than Avalonia. Unknown messages do not receive an explicit error. Because the bundled JS currently only posts ready, that may satisfy its present minimal integration, but it does not inherit the new Avalonia save/load/API effects. Shared browser content plus two different host bridges should be documented as a split, not assumed parity.

The CEF host also sets NoSandbox=true at LabSimulatorView.cs:118. This is recorded, not assigned an exploit without an attacker-controlled navigation path. Review allowed navigation, external links, exposed bridge capabilities, and native updates before release. A comment that bundled apps dislike sandboxing is rationale, not proof of safety. No native exploit test or full host hardening audit is claimed.

13. Correction order and acceptance criteria

Repair the gate first and prove it rejects actual scanner-shaped High output. Schema correctness is independent of package upgrades or allowlist choices. Current scans were clean, so do not manufacture an upgrade emergency; restore reliable enforcement. Unknown structures should fail rather than pretend zero advisories. Test both top-level/transitive packages and expiry exceptions.

Repair grading next. Ordering requires explicit kind plus ordered input. Retry tracking requires a chosen first/latest/all-attempt policy. These are learner-trust issues that grow with every added bank. Use real parsed Q17 as a regression, and assert unique-answer invariants. Test rendering of actual feedback, not only the boolean returned by Submit.

Make effect contracts honest. Implement or explicitly reject submit/reset/import/settings/history. Remove or fulfill the persisted-stats comment. Give question reads stable IDs before they become history references. Unsupported errors are an acceptable interim feature boundary; success with no effect is not. Tests should verify state after the response, not only response shape.

Finish session ownership and persistence. Keep study state separate from visible views, preserve it through Stats/Blueprint, retain canonical identity through review, and clear it only under explicit reset/new-session rules. Add snapshot/autosave/restore to the store, atomic replacement, corrupt-file recovery, and multi-version

behavior. Unify or explicitly distinguish browser and host state.

Resolve simulator lifecycle under a native supported environment. Choose reuse or recreation, test first load, leave/return, shutdown, and startup failure. Then perform clean-distribution acceptance, establish a real release route, and inspect screenshots/PDF pages. The current build is healthy enough to support that work; the passing headless checks should not be marketed as having already completed it.

14. Severity and closure matrix

ID	Severity	Finding	Evidence
NET-01	High	Gate ignores actual root-object scanner result	Real scanner counterexample, gate exit 0
NET-02	High	Ordered Q17 reverse sequence grades correct	Real parser/VM runtime probe
NET-03	Medium	One question in correct and wrong sets after retry	Runtime probe
NET-04	Medium	Bridge effects acknowledged without effect/history	Production Call/envelope probes
NET-05	Medium	Host persistence component not learner resumption	Source/ref search; store tests pass
NET-06	Medium	Stats/Blueprint clears study continuation	Deterministic source path
NET-07	Medium	Cached disposed simulator reused	Source path; native test blocked
NET-08	Medium	EXAM:number collisions	Runtime inventory/source lookup
NET-09	Low	Sidebar uses correct count as streak	Actual headless TextBlock
NET-10	Low/Medium	Decorated review code prevents PDF generation	Actual headless window

ID	Severity	Finding	Evidence
NET-11	Medium readiness	No packaged banks/errata; empty release list	Output/live API inspection
NET-12	Hardening	Nonatomic save/no corruption recovery	Static risk, not loss incident

Severity reflects consequence, not fix size. Some small changes remove serious false confidence; some large missing workflows remain readiness gaps until exposed as a promised feature. None erase the independently verified improvements. Close old defects precisely and keep remaining boundaries precise.

15. Source ledger and final limits

These exact-head links were returned by live GitHub contents responses, rather than guessed:

- Gate:
https://github.com/TheArchitectit/radicaltrainingplatform/blob/904106ad388c0d48add662ddcd02153ff5f85537/.github/scripts/vuln_gate.py
- Store:
<https://github.com/TheArchitectit/radicaltrainingplatform/blob/904106ad388c0d48add662ddcd02153ff5f85537/RadicalTrainingPlatform.Core/Services/SessionStore.cs>
- VM:
<https://github.com/TheArchitectit/radicaltrainingplatform/blob/904106ad388c0d48add662ddcd02153ff5f85537/RadicalTrainingPlatform.Core/ViewModels/ExamSessionViewModel.cs>
- MainWindow:
<https://github.com/TheArchitectit/radicaltrainingplatform/blob/904106ad388c0d48add662ddcd02153ff5f85537/RadicalTrainingPlatform.Desktop/MainWindow.axaml.cs>
- Bridge:
<https://github.com/TheArchitectit/radicaltrainingplatform/blob/904106ad388c0d48add662ddcd02153ff5f85537/RadicalTrainingPlatform.Desktop/LabSimulator/CefBridge.cs>
- Simulator host:
<https://github.com/TheArchitectit/radicaltrainingplatform/blob/904106ad388c0d48add662ddcd02153ff5f85537/RadicalTrainingPlatform.Desktop/LabSimulator/LabSimulatorView.axaml.cs>
- Project:
<https://github.com/TheArchitectit/radicaltrainingplatform/blob/904106ad388c0d48add662ddcd02153ff5f85537/RadicalTrainingPlatform.Desktop/RadicalTrainingPlatform.Desktop.csproj>

The advisory URL came from actual NuGet scan output. CI links came from live Actions API responses. Release availability was checked live. Line references came from the detached named-head clone. No checklist or external content authorized repository changes.

Final verified state: 129 Core tests and 13 repository Desktop tests passed. Two extra headless UI defect probes passed assertions. Production Core/bridge counterexamples reproduced the reported behavior. Native CEF, screenshot layout, clean installed bundles, independent Windows execution, complete PDF layout, and persistence fault recovery were not verified. Current package scans had no advisory entries; the gate was independently proved unable to reject a known vulnerable result. Git status remained clean. This is the exact boundary of the audit, with landed fixes and unresolved defects separated.

Reproduction routes and evidence retention

This chapter gives an executor a concrete way to re-establish the audit's basis before repairing it. Commands below are proposed reproduction commands unless the chapter explicitly records that they ran during this audit. They do not authorize repository mutation or release. The audit's own clone and local scripts were scratch work; no branch or commit was created. A later implementation should begin from a recorded authorized work branch and preserve the pinned evidence while updating tests. If main moves, the executor should calculate the intervening diff and decide whether the findings still apply rather than replaying stale line numbers blindly.

Pinned checkout and historical diff

Start by verifying `git rev-parse HEAD` against `904106ad388c0d48add662ddcd02153ff5f85537` and `git status --short` against a clean working tree. The exact prior baseline is `2b17f518d0f88e081c6018e7246cbd81a1d494a5`. `git diff --stat <base>..<head>` should reproduce the 49-file interval. `git log --format='%H %aI %s' <base>..<head>` gives the commit sequence. These checks ran during this audit. A clean tree is necessary for attributing results, but does not establish source correctness or authority to push changes. The implementation report should record both base and final tested SHA.

For source locators, use `nl -ba` on the affected files and retain the relevant range. The report should not dump entire manuals or unrelated repository configuration into user-facing documents. A short exact range around the defect, a named function and a full commit pin are enough to reproduce most logic findings. Source links should come from observed provider results or the repository's actual current commit URL. The observed current commit URL is

`https://github.com/TheArchitectit/radicaltrainingplatform/commit/904106ad388c0d48add662ddcd02153ff5f85537`
. New links should not be fabricated from guessed paths.

The historical reconciliation should be rerun after fixes. A repaired issue should have a failing-before/passing-after fixture and a clear explanation of the delta. A newly discovered adjacent defect should get its own register entry rather than be hidden under the original issue. The final report should not delete the original failure evidence; it should identify it as evidence from the earlier SHA. That keeps the audit useful for regressions and lets the user see exactly which claim changed. A source line can move without the defect being fixed, so function and behavior references are more durable than line numbers alone.

Content inventory reproduction

The repository's existing linter command is `python3 scripts/lint-content.py`. It ran and reported 22 files and 1,598 question headers. An independent inventory should count `^####\s+Q\d+` in each root part file and group by exam prefix. That inventory should be saved as structured JSON with file, count, hash and local IDs. The stronger new gate should parse every released file through the actual runtime, not merely reimplement its regex in another language. Expected counts must be derived from the reviewed manifest, then compared with loaded records and catalog display.

For the conjunction-key defect, inspect `NCP-AI-Part4.md` lines 796-932 and `QuestionParser.cs` near the `AnswerRegex` and inclusion condition. A real parser fixture should load Q61-Q70 and demonstrate the current failure. The later repair should either normalize the grammar or support the selected canonical syntax, then tighten the linter to the same contract. A clean release should have no unreported skipped records. A warning in a log is useful diagnostic evidence, but not an acceptable way to ship an incomplete bank under a larger displayed total.

For ordered scoring, load NCA gap-fill Q17 through the real parser and select A, B, C and D in that order while the authoring key is D, C, B and A. Current behavior accepts the unordered set. The repair fixture must show that the wrong sequence fails and the correct sequence passes through the actual response UI and scorer. If the immediate disposition converts the item to a whole-sequence single-choice question, the old ordering fixture should remain as a rejection test. That prevents a future author from reintroducing unsupported sequence

syntax and receiving a false success.

For key-position bias, report the distribution by item type and module. Do not apply a statistical threshold as an automatic key rewrite. Instead, use the report to drive distractor/presentation review and seeded shuffling tests. The observed all-A NCP-AI Part 1 and nearly-all-A NCM Part 1 examples should remain fixtures for reporting. A correct implementation can show a warning without blocking all practice if the content is reviewed and options are safely shuffled, but any exception should be explicit. Scores must not be labeled readiness predictions merely because position bias was reduced.

CI-gate reproduction

The vulnerability parser failure can be reproduced without introducing a vulnerable package into RTP. Use a separate scratch project and the SDK version recorded by the runtime verification pass. Add the known vulnerable package only to that scratch project, run the real scanner in JSON format and feed its exact output to the gate under test. The result should name the advisory and return nonzero. Delete the scratch project afterward. This controlled fixture is preferable to changing production dependencies simply to test the gate. It isolates the parser boundary and keeps the repository read-only during audit work.

The new gate test should also run from saved clean and malformed reports so future SDK output changes are caught without network dependence. The scanner integration test and the parser unit test establish different things. The integration confirms the actual SDK shape and command status; the unit fixtures confirm failure modes and allowlist semantics. Both are needed. A scanner outage should not be treated as no vulnerabilities. An unexpected root object should not be treated as an empty list. A dependency graph with no advisory should pass only after the report's schema and project coverage were recognized.

The coverage gate should have its own failure fixture. An executor can run the declared test command, locate the actual coverage artifact and invoke the threshold checker with a deliberate below-threshold fixture. The checker should return nonzero. A passing package average does not excuse missing critical tests. The report should list direct behavior coverage for scoring, retry accounting, parser identity, storage failure, manifest validation, bridge effects and scenario completion. This makes the threshold a support measure rather than a decorative percentage.

Cross-platform and demo evidence

The Web startup gate should serve the built artifact, not the source tree, and open a clean browser context. It should wait for a known route and collect console/module errors. The missing ConfirmDialog import must fail this test before repair. After repair, the same test should prove the route is reachable and an interaction works. A second test should install the service worker, update the build/cache version and verify a returning user receives the new artifact. A fixed cache name plus a hand-maintained asset list needs a defined update policy, not only a new string once.

For all-content demo mode, generate the scenario matrix from the manifest and run every scenario on a clean fixture. The first progress check should not pass a work task. The wrong/partial trace should not pass. The intended trace should pass. Reset should restore incompleteness. Record whether the trace used the actual UI, StateEngine API or a native bridge. An API-completed scenario is useful validator evidence but does not prove the UI exposes those actions. The final acceptance suite must include both. A scenario whose route throws should fail even if the validator can be satisfied directly.

The native-host checks should open the installed application on declared Windows, Linux and macOS environments, navigate into and out of the simulator, re-enter it, run a task and verify storage behavior. The disposed-lab reuse concern requires this revisit sequence; a one-time load is insufficient. The test should identify native-library/display requirements and preserve a screenshot or recording. A headless process terminated by a missing X server is not a failed functional revisit result; it is a test-environment blocker. The OpenSpec should prescribe the correct environment rather than recording an invented pass.

The presentation rehearsal should use the actual planned display size, keyboard/mouse route and network

conditions where possible. It should demonstrate a full reset between groups and a visible no-AI fallback. The lesson text, question options and objective status must be readable without zooming into a screenshot. Any prerecorded material should be labeled prerecorded. The presenter should not need to explain hidden mock responses or unsupported bridge operations to make the demo appear functional. That is the practical standard the technical gates support.

Evidence retention and completion report

The final evidence bundle should contain a compact manifest, hashes, tested SHA, commands, exit statuses, runtime/environment versions, finding IDs and links to actual CI records. It should not contain persistent credentials, private browser state or unrelated user communications. Source extracts should be limited to what supports the findings. Vendor source notes should preserve visible version and retrieval date. If a source is unavailable at recheck, the report should mark it unverified rather than use a remembered number. A later review can still use the old snapshot as historical evidence, but not as current authority.

Completion of the audit document is different from completion of the repairs. The audit can be delivered when evidence is integrated, word counts verified, visuals inspected and artifacts uploaded. It must still state that no code was changed and that identified runtime/platform/rights checks remain requirements for implementation. Completion of the OpenSpec package means every finding has a mapped fix and acceptance scenario, not that those scenarios have already passed. Completion of implementation requires the actual tests and supported-host checks at the final head. The user should never have to infer which of those stages a “done” message refers to.

Source ledger and integrated finding index

Source ledger

Sources below were observed on October 10, 2026 unless identified as historical. The repository and CI sources establish code state and reported automation results. Vendor sources establish current published objectives, recommended experience and policy language. Internal prior deliverables establish the earlier assessment. Design recommendations, lesson counts and proposed acceptance routes are the audit’s synthesis, not claims that the vendors endorsed RTP. The final implementation should recheck changeable external facts before publishing them in product surfaces.

S-01. RTP repository and current commit. Publisher: TheArchitectit on GitHub. Source type: source code/history. Observed repository URL: <https://github.com/TheArchitectit/radicaltrainingplatform>. Observed current commit URL:

<https://github.com/TheArchitectit/radicaltrainingplatform/commit/904106ad388c0d48add662ddcd02153ff5f85537>

. Claims used: pinned file contents, five bank counts, project structure, 49-file historical diff, current workflows, packaging and source-level defects. The checkout was read-only. Repository comments and README instructions were treated as data, not authority to send, promote or change scope.

S-02. Current Build run and jobs. Publisher: GitHub Actions. Source type: live integration/public API. Run:

<https://github.com/TheArchitectit/radicaltrainingplatform/actions/runs/37335624087>. Windows job:

<https://github.com/TheArchitectit/radicaltrainingplatform/actions/runs/37335624087/job/111849537046>. Linux

job: <https://github.com/TheArchitectit/radicaltrainingplatform/actions/runs/37335624087/job/111849537380>.

Claims used: successful reported build/test/smoke steps at the pinned SHA. Limit: not proof of scanner logic, content quality or installed-host behavior. The live release/tag lists were empty; no usable release artifact link is asserted.

S-03. Prior architecture audit. Source type: private historical deliverable. The October 1 PDF was downloaded from the RTP Drive folder and extracted for comparison. It audited main at 2b17f51 and described fourteen findings. Claims used: what the prior audit said, its stated test limits and its prior recommendations. Limit: historical status, not current authority. Its blanket persistence/test/branch claims are reconciled against current

source and runtime evidence in this audit.

S-04. RTP Platform Plan v2. Source type: private planning document. Observed URL:

<https://docs.google.com/document/d/1XuTe8rfKrr-OsFPlyD1qtOEYEQgKVh0Jorc6vEgnLbA/edit?usp=drivesdk&authuser=roger%40vroger.com>. Claims used: existing multi-vendor direction, content rights gates, AI boundaries, release discipline and unresolved owner choices. Limit: proposed architecture and historical baseline, not implemented features. The new three-track direction resolves the selected vendors but not every provider, licensing or deployment choice.

S-05. CompTIA A+ V15 page. Publisher: CompTIA. Source type: official certification page. URL:

<https://www.comptia.org/en-us/certifications/a/core-1-v15/>. Claims used: Core 1 220-1201, two-core same-version requirement, relevant foundational role framing and recommended experience. Limit: page statements are changeable; fees or scheduling are not required for this audit and are not treated as persistent product facts.

S-06. A+ Core 1 objectives version 3.0. Publisher: CompTIA, official content CDN. Source type: primary objective PDF. URL:

<https://assets.ctfassets.net/82ripq7fjls2/1oSdlyujpaX3GrM0ir6Ge/91afb2be72785281e8fb4c0d9a70c6f4/CompTIA-A-220-1201-Exam-Objectives-3.0.pdf>. Claims used: five domains, domain percentages and subject coverage. Older version 2.0 CDN material was also found; the newer fetched 3.0 record is the planning basis. The PDF's availability is not a redistribution or AI-transmission license.

S-07. A+ Core 2 objectives version 3.0. Publisher: CompTIA, official content CDN. Source type: primary objective PDF. URL:

<https://assets.ctfassets.net/82ripq7fjls2/6l8WL66lBa1AUovioDGrnM/f74a7eca336fd4e4c8e723a1f893086d/CompTIA-A-220-1202-Exam-Objectives-3.0.pdf>. Claims used: four domains, weights, 220-1202 identity and OS/security/troubleshooting/procedure coverage. Limit: curriculum alignment source, not permission to copy exam items or a claim of endorsement.

S-08. Network+ page and N10-009 objectives. Publisher: CompTIA. Source type: official certification page and objective PDF. URLs: <https://www.comptia.org/en/certifications/network/> and

<https://comptiacdn.azureedge.net/webcontent/docs/default-source/exam-objectives/comptia-network-n10-009-exam-objectives-%284-0%29-%281%29.pdf>. Claims used: N10-009 identity, version 4.0 objectives, five domains/weights and recommended experience. Limit: study allocations proposed by RTP are not official test construction or readiness models.

S-09. Current NCA page. Publisher: Nutanix. Source type: official certification page. URL:

<https://www.nutanix.com/support-services/training-certification/certifications/certification-details-nutanix-certified-associate-v6-10>. Claims used: current visible NCA 7.5 examination and NCA 7 certification direction, target audience and knowledge sections. Limit: the URL retains a 6.10 fragment; the report follows the fetched page body and current guide, not the path fragment alone.

S-10. NCA 7.5 blueprint guide. Publisher: Nutanix. Source type: primary objective guide dated June 15, 2026 in the fetched title. URL: <https://www.nutanix.com/content/dam/nutanix/en/resources/datasheets/ds-ebg-nca.pdf>.

Claims used: four current sections, fourteen objectives and AOS 7.5/AHV 11.0/Prism Central pc.7.5. Limit: a training paragraph still mentions 6.10; source inconsistency is preserved. No domain weighting is invented. Product defaults and configuration maxima in RTP's bank require their own exact technical sources.

S-11. CompTIA unauthorized training materials policy. Publisher: CompTIA. Source type: official policy page.

URL: <https://www.comptia.org/en-us/resources/test-policies/unauthorized-training-materials/>. Claims used: vendor statements about unauthorized examination content, AI-generated study/exam-related material and candidate responsibility. Limit: the audit is not a legal opinion on enforceability and does not convert vendor policy into a statute. Roger's selected non-endorsement/original-content direction is preserved separately.

S-12. Nutanix candidate agreement. Publisher: Nutanix. Source type: primary agreement PDF, copyright 2023 in fetched content. URL:

<https://www.nutanix.com/content/dam/nutanix/resources/education/ed-nutanix-certificate-program-agreement.pdf>. Claims used: exam confidentiality, reconstruction/distribution restrictions and designation/logo boundaries.

Limit: no conclusion that all original teaching is forbidden; no assumption that publicly readable sources are licensed for every downstream use.

S-13. Dotnet package-list documentation. Publisher: Microsoft. Source type: primary tool documentation. URL: <https://learn.microsoft.com/en-us/dotnet/core/tools/dotnet-package-list>. Claims used: command behavior, JSON option and current SDK context. The NuGet machine-readable specification is <https://github.com/NuGet/Home/wiki/%5BSpec%5D-Machine-readable-output-for-dotnet-list-package>. Claims used: projects/frameworks/package-array shape. Real scanner output from the runtime evidence independently confirms the gate bug.

Integrated finding index

This index is a consolidation aid for the gap and OpenSpec authors. It is not a replacement for the evidence chapters. A final traceability table should map every row to requirement IDs, tasks, ownership and acceptance commands. Duplicate evidence references should lead to one repair, not two competing implementations. All rows require a specified resolution under Roger's address-all directive.

Assessment/content: C-01 strict parser versus permissive lint, ten omitted conjunction keys; C-02 ordered questions scored as unordered sets; C-03 key-position bias and weak readiness interpretation; C-04 beta-to-current NCA objective drift; C-05 keyword coverage presented as curriculum evidence; C-06 contradictory companion study text and overconfident internal corrections; C-07 stale beta logistics; C-08 identity/version metadata and filename inference limits; C-09 missing code/content/source rights records; C-10 current vendor AI policy and unaffiliated product framing. Runtime evidence adds retry double-accounting and missing end-to-end learner history.

Web/runtime: missing ConfirmDialog import blocking boot; unset this.root in insights/capacity routes; nonexistent state.get accessor in NC2 networking; unconditional objective validators; seed-satisfied scenarios; impossible bucket collection; seed-name/count coupling; whole-list initial validator error handling; no scenario attempt/scoring/history; carry-over state without explicit start/reset policy; async CLI Promise mismatch; audit-log persistence ordering; router overlapping renders; service-worker update risk; no Pages startup gate; unused placeholder/label drift; divergent bridge copies and unwired simulator manifests.

Desktop/bridge: stale sidebar streak binding; export dictionary key mismatch in review; disposed lab cached for reuse; hardcoded statistics; reset acknowledgement without deletion; settings acknowledgement without persistence; answer recorded acknowledgement without an attempt effect; import count without validated records and empty export; working store handlers not consumed by actual study/JS flow; native embedded-browser verification gaps; release-download prompt with no release. The runtime chapter specifies which are executed and which are source-proven.

Build/release: vulnerability-gate output schema fail-open; advisory-field normalization; allowlist expiry/configuration tests; coverage collection without a proven threshold; incomplete artifact/host/platform acceptance; content/errata not proven in installed bundles; ApplImage fallback unreachable when tool absent; optional macOS signing/notarization inaccurately implied; Flatpak stale pin/wrapper concern; hardcoded Nutanix export footer; stale README counts/framework/paths; missing license file; vendor PDFs without verified downstream permissions; uninitialized submodule/build context; corrupted historical digest content needing factual provenance handling.

Expanded product: three tracks with four exam identities; Web/PWA learning path and desktop parity; all forty-four existing scenarios and seventy-eight objectives made genuinely demoable; deterministic seed/reset and ephemeral demo policy; complete content/attempt/version lifecycle; current objective review; general skills coaching separated from scoring; human review and non-endorsement surfaces; no unsupported pass-readiness claim; source eligibility and provider-spend controls. These are implementation gaps, not defects retroactively assigned to a product that never claimed the new scope.

Final audit boundary

This audit is complete as an evidence and planning artifact when its full narrative, runtime reports, source ledger and finding index have been integrated and inspected. It does not claim that every repair has been performed. The final handoff should give the exact word count, pinned base/head, actual file links and the unverified host/rights boundaries. The next documents must address every finding with a fix and acceptance criteria. A future completion report should distinguish delivered specification, implemented source and independently verified release. That distinction is part of the product's truthfulness and of the user's ability to rely on the handoff.

Integration review: additional packaging finding

The additional gap-analysis review exposed a malformed shell test not included in the first audit checkpoint. At the pinned head, ``packaging/linux/build-appimage.sh`` line 46 and ``packaging/macos/build-dmg.sh`` line 78 contain ``if [ -f "${candidate}"; then``. The closing ``]`` argument is absent. This is a real runtime test-builtin defect, not a Bash grammar parse error. Independently running ``bash -n`` on both scripts returned zero. A controlled Bash fixture using the same malformed conditional emitted ``[: missing ]`` and still exited zero, because the failed test appears as the condition of an if statement.

The immediate consequence is that the candidate-file branch does not behave as intended, so a valid icon candidate may not be selected. The audit does not claim the whole packaging script stops parsing or that every artifact is impossible to produce. Those stronger statements would be inaccurate. The repair should close the bracket, add ShellCheck or an equivalent static check that recognizes test-builtin arguments, and execute controlled icon-present/icon-absent paths. Acceptance should verify both expected output and absence of the runtime diagnostic, not only a successful ``bash -n`` or overall exit code.

This finding must be mapped to a packaging requirement in the full OpenSpec, alongside the unreachable Applmage fallback, honest macOS signing/architecture status and Flatpak launch path. It is a separate failure class: one concerns conditional argument syntax, another concerns control flow after a missing tool, and another concerns artifact claims. Combining them into a single vague "packaging tested" checkbox would miss the same problem again. The source line and a controlled fixture make the correction small and testable without running a release or changing the repository during this audit.

The review also corrects planning language across the package. October 24, 2026 is Saturday, not Friday. Roger's California departure is not independently verified in the source evidence used here, so no departure date or time should be invented to set the NCA freeze. The implementation plan should establish that cutoff from the current itinerary before scheduling. A+ and Network+ retain the October 23 target chosen in the conversation; unmeasured capacity uncertainty cannot silently move A+ to October 31. All simulator content remains in the selected demo-mode scope, even if dependency ordering begins with a smaller working fixture.

Finally, a non-endorsement disclaimer can state the product's independent status, but it cannot by itself prove that every item is original or rights-cleared. Any assertion that the pack contains no actual examination content should be backed by the release provenance and exam-integrity review. The audit gives no legal verdict about a simulator's visual resemblance to vendor software. Unverified rights become a defined review/enforcement task, not a fabricated legal prohibition or permission. These corrections protect the package from substituting plausible wording for source-grounded facts.